

Tu Ford te avisa antes

Predicción temprana de la degradación del filtro de partículas en los diésel conectados de Ford

Informe final del equipo SOG

Gonzalo · Santino · Octavio

En una página

- **Qué hicimos.** Un modelo que, cada 500 km, lee los últimos 1.000 km de telemetría de un auto y marca a los que van camino a una falla del filtro de partículas (DPF). Nunca ve los 500 km previos al evento. Alrededor del modelo armamos un circuito que convierte cada alerta en una acción: un consejo al conductor o un llamado del concesionario. Los textos los redactan agentes de IA y los controla un verificador en código.
- **Cuánto detecta [test].** Sobre 111 autos que estuvieron bajo llave todo el proyecto (103 con datos, 32 fallados), la GRU finalista avisa a ~30 % de los autos que fallan con 5 % de autos sanos con falsa alarma, a ~40–50 % con 10 % y a ~50–60 % con 20 %. El azar avisa a 7 % con 5 %.
- **Con cuánto margen [test].** La primera alerta llega con una mediana de ~4.600 km, unos 3 meses, antes de la falla registrada.
- **Lo que no prometemos.** No le gana con evidencia a la regla “país × motor”; sabe sobre todo *qué auto* y poco *cuándo*; con 32 fallas los intervalos son anchos; la prevalencia real se mide en un piloto; no predecimos la fecha exacta.

Índice

Resumen ejecutivo	3
Cómo leer los números de este informe	4
Cómo responde este informe a los criterios de Ford	4
1. Descripción general del desafío	6
1.1. Título del desafío: qué evento hay que anticipar	6
1.2. Contexto: la experiencia del cliente	6
1.3. El desafío: predecir la degradación debida al uso	6
2. Contexto del proceso	7
2.1. El proceso actual: la detección es reactiva	7
2.2. Ubicación en el flujo: posventa	7
3. Definición del desafío	8
3.1. Desafío específico: anticipar el momento, sin hacer trampa	8
3.1.1. La pregunta: un panel de cortes con gap de blanking	8
3.1.2. Las reglas que no se negocian	9
3.1.3. Cómo medimos, y por qué cambiamos de métrica	9
3.2. Impacto operativo	10
4. Datos técnicos relevantes	10
4.1. Las tres tablas: lo que prometía el anexo y lo que llegó	11
4.2. Equipos o sistemas existentes: el histórico por vehículo	11
4.3. Restricciones técnicas: anonimización y renombres	12
4.4. Lo que encontramos: las trampas de los datos	12
4.5. Limpieza, faltantes y atípicos	14
4.6. El universo del estudio y el holdout	15
4.7. Las features: qué entró, qué no y por qué	16
4.8. Qué dicen los datos: el análisis exploratorio	16
4.9. Consistencia y trazabilidad del pipeline	19
5. Expectativas de solución	19
5.1. Objetivo de mejora: anticipar con el mayor margen posible	19
5.1.1. El recorrido: más de 40 modelos, la misma cuenta	19
5.1.2. El finalista: una red que lee los viajes en orden y recuerda	21
5.1.3. El resultado, en autos que nunca vio	22
5.1.4. Cuánto antes: el margen medido	23
5.1.5. Una variante que ganaba en desarrollo y no se sostuvo	24
5.1.6. Límites	25
5.2. Criterios de evaluación	25

5.2.1. Viabilidad técnica	25
5.2.2. Costo	27
5.2.3. Carácter innovador	28
5.2.4. Calidad del tratamiento de datos	29
5.2.5. Claridad para mostrar los datos	29
5.2.6. Explicabilidad del modelo	29
5.2.7. El pitch	31
6. Información complementaria	31
6.1. Documentación disponible	31
6.2. Referentes técnicos	31
6.3. Lo que solo Ford tiene: preguntas abiertas	32
6.4. Trabajo futuro	32
6.5. Conclusiones	32
A. Reproducibilidad: comandos y configuraciones	33
B. Auditorías de leakage	34
C. Todos los modelos en test	35
D. Diccionario de features	36
E. Las tablas de Ford: anexo contra archivos	38
F. Anexo técnico: el PR-AUC por fila y el techo de cohorte	39

Resumen ejecutivo

El problema. El filtro de partículas (DPF) de los diésel se tapa según cómo se usa el auto: viajes cortos, motor que no llega a temperatura, mucho tiempo detenido con el motor en marcha. Hoy Ford se entera tarde, cuando el testigo ya se prendió o el auto ya está en el taller. El desafío pide anticipar ese evento con la telemetría del auto conectado, con la mayor cantidad de tiempo o kilometraje posible.

Lo que encontramos en los datos. La telemetría sí deja huella, pero viene con trampas que, sin corregir, hacen que cualquier modelo parezca mejor de lo que es. Las dos cohortes (fallados y sanos) se muestrearon en períodos de producción distintos: la fecha de fabricación sola ya las separaba. La fecha registrada del evento cae unas dos semanas después de la intervención en el taller. El contador de regeneraciones se apaga para toda la flota el 25-05-2026. Hay 13 autos duplicados bajo dos códigos. La segunda entrega de Ford trae cinco trampas de formato. Corregimos todo esto en un pipeline declarativo y trazable (§4). Comparando contra sanos del mismo mercado, los autos que van a fallar pasan cada vez más tiempo en ralentí y con el motor frío a medida que se acerca el evento.

Lo que hicimos. Cada 500 km preguntamos: mirando los últimos 1.000 km, ¿este auto falla en los próximos 3.000? Dejamos 500 km ciegos antes de la falla (el *gap de blanking*), porque sin ese margen estaríamos detectando lo que el tablero ya detecta. Medimos más de 40 modelos con la misma cuenta: la detección por auto con un presupuesto fijo de falsas alarmas, siempre al lado del azar y de la regla “país × motor”. El finalista es una red recurrente (GRU) que lee la secuencia de viajes y de señales del filtro, más país, motor y modelo, en un ensamble de tres semillas.

Cuánto detecta, en autos que nunca vio [test]. Sobre el holdout (111 autos, 103 con datos, 32 fallados): **10 de 32 al 5 % de falsas alarmas** (31,2 %, con un azar de 7,0 %), 46,9 % al 10 % y 62,5 % al 20 %. Leímos la misma configuración tres veces en test, así que el resultado se cita en rango: **~30 % al 5 %, ~40–50 % al 10 % y ~50–60 % al 20 %**. Con el umbral fijado antes, en los autos de desarrollo, detecta 20 · 34 · 57 % con 5,8 · 8,9 · 18,2 % de falsas alarmas reales: el umbral elegido sin ver el test cumple su presupuesto. Le duplica la detección media a los modelos tabulares y de supervivencia.

Con cuánto margen [test]. La primera alerta llega con una mediana de ~4.600 km (p25–p75: 2.400–9.300 km), unos 100 días antes de la falla registrada. El puntaje sube a medida que se acerca el evento.

Qué pasa después de la alerta [dev]. Una política determinista (la define Ford) decide si corresponde un consejo al conductor o un llamado del concesionario. Dos agentes de IA redactan los textos y un verificador en código los controla: ningún LLM produce un número, un puntaje ni un diagnóstico. La demo reproduce semana a semana la flota de desarrollo con una perilla de falsas alarmas de 5 a 20 %.

Cuánto cuesta y cómo escala. No hacen falta sensores nuevos: usa solo resúmenes de viaje y mensajes del filtro que el auto ya manda. Según nuestra estimación (a validar con Ford), operarlo para un millón de autos cuesta ~USD 0,07–0,10 por auto y por año. La GRU tiene unos 3.900 parámetros y corre en CPU.

Lo que no prometemos (y conviene decirlo antes de que lo pregunten)

- **No le gana con evidencia a la regla “país × motor” [test]:** +27,3 puntos de detección media al 5–20 %, IC95 [–0,8; 46,1]. Queda en el límite y no se cita como victoria.
- **Sabe poco del cuándo:** la auditoría (a') da +0,001 [dev]. Lo que el modelo sabe es sobre todo *qué auto* va a fallar. Lo que sí se ve es que el puntaje sube hacia el evento.
- **Con 32 fallas en test, cada auto mueve 3 puntos** y los intervalos son anchos.
- **El test se leyó más de una vez** con la misma configuración (§5.1.3). Por eso se cita un rango y no

un número.

- **La muestra está enriquecida en fallas:** la prevalencia real de la flota no la conocemos y se mide en un piloto en sombra.
- **No predecimos la fecha exacta:** marcamos con margen.

Cómo leer los números de este informe

- **[test]:** medido sobre los 111 autos del holdout, sorteados el 26-09 antes de mirar ninguna feature contra la etiqueta. De ellos, 103 tienen cortes en el panel: 32 fallados y 71 sanos. Cada modelo se entrenó con todo el conjunto de desarrollo.
- **[dev]:** medido con validación cruzada agrupada por vehículo sobre los 446 autos de desarrollo (5 folds × 3 repeticiones). La demo, la explicabilidad y las auditorías son de dev.
- **Detección al X %:** de los autos que fallan, cuántos reciben una alerta antes de la falla, con un umbral fijado para que solo el X % de los autos sanos tenga una falsa alarma. Una alerta son dos cortes seguidos sobre el umbral.
- **Azar:** el mismo puntaje permutado entre filas, 200 veces, conservando el largo del historial de cada auto. Se reporta su media y su percentil 95.
- **Celda mercado × motor:** la tasa de fallas de la celda del auto (calculada sin él), usada como puntaje fijo. No mira ningún viaje y es el piso que cualquier modelo tiene que superar.
- **Nada de accuracy.** Con ~6 % de filas positivas, “nadie falla” daría 94 %. El PR-AUC por fila va solo al anexo técnico, al lado del techo de cohorte (Anexo F).

Cómo responde este informe a los criterios de Ford

El informe sigue las siete secciones del documento del desafío. En cada una contamos qué entendimos, qué encontramos en los datos y qué hicimos. La tabla 1 ubica cada criterio de evaluación de la sección 5.2 y el objetivo de mejora de la 5.1.

Tabla 1: Criterios de evaluación de Ford (§5.1 y §5.2 del desafío) y dónde se responden.

Criterio	Qué mostramos	Dónde
Objetivo 5.1: anticipar con el mayor tiempo o kilometraje posible	Anticipación medida en test (~4.600 km, ~100 días de mediana), con el gap de 500 km que impide la detección reactiva; el puntaje sube hacia el evento	§5.1.4, §3.1.1
Viabilidad técnica	Modelo medido en un holdout congelado; GRU chica en CPU; mismo código de features en entrenamiento y producción; demo funcionando; plan de despliegue por etapas	§5.2.1, §5.1.3
Costo	~USD 0,07–0,10 por auto y por año para un millón de autos, sin sensores nuevos; escenarios de costo de falla como reporte	§5.2.2
Carácter innovador	Una métrica que separa <i>qué auto</i> de <i>cuándo</i> ; red recurrente sobre la ventana en km; agentes de IA con verificador en código; la perilla de falsas alarmas	§5.2.3
Calidad del tratamiento de datos: limpieza, faltantes y atípicos	Atípicos con topes físicos; ralentí separado de los viajes; faltantes estructurales tratados como tales; marcador cortado reconstruido	§4.5
Calidad del tratamiento de datos: consistencia y trazabilidad del pipeline	Correcciones declaradas en YAML; universo y holdout congelados; 13 clones colapsados; reglas antileakage; auditorías obligatorias; cada número con su comando	§4.9, Anexos A y B
Claridad para mostrar los datos	Figuras de este informe; demo de producto; dashboards del EDA y de explicabilidad; curva de detección contra falsas alarmas con el azar	§5.2.5
Explicabilidad del modelo	En qué se apoya la GRU (permutación de señales); el porqué que ve el conductor (comparación contra la flota sana, filtrada por la física del DPF)	§5.2.6
Pitch	La misma historia en siete bloques, con los mismos números y los mismos límites	§5.2.7

1 Descripción general del desafío

1.1 Título del desafío: qué evento hay que anticipar

Ford pide una *metodología de inteligencia artificial para la predicción temprana de eventos de degradación en la eficiencia de combustión e ingreso de oxígeno al motor, a partir de datos de vehículos conectados.*

Qué entendimos. En un diésel moderno, el componente que se degrada por el uso y que afecta a la vez la combustión y el flujo de gases es el sistema de postratamiento, y en particular el filtro de partículas (DPF). El DPF atrapa el hollín del escape y, para no saturarse, lo quema periódicamente: eso es una *regeneración*. La regeneración necesita temperatura, es decir, un tramo de ruta con el motor caliente. Con viajes cortos, la regeneración empieza y no termina, y el hollín se acumula. Un filtro cargado sube la contrapresión del escape. Si sigue así, el auto entra en un modo de protección con potencia limitada y termina en el taller.

Qué encontramos en los datos. El dataset confirma que el evento es del filtro, aunque las columnas vienen renombradas (§4.3):

- La tabla `DynamicInformation` trae un mensaje de la ECU con nueve niveles del tipo “Air Filter Full”, “Air Filter Overloaded” o “Cleaning Automatically Air Filter”, y una variable de acumulación en escala 0–100 que sube con el uso y se descarga de golpe. Esa es la dinámica de un filtro que se carga y regenera.
- `TripSummary` no trae las columnas `DieselParticulateFilterStart/End` del anexo, pero sí trae cuatro que el anexo no menciona: `AirFilterStart/End` y `AirRegenerationStart/End`. Son la foto, al inicio y al final de cada viaje, de las mismas dos variables de `DynamicInformation`. Coinciden en el 99,97 % y en el 99,8 % de los pares alineados.
- Que esas columnas *sean* el DPF es la explicación que mejor encaja (ocupan su lugar en la tabla, su dinámica es la de un filtro y el anexo avisa que las variables vienen renombradas), pero Ford no la confirmó. Por eso en el informe las llamamos “nivel de acumulación del filtro”.

Hay una pregunta que los datos dejan abierta. La ceniza en un DPF es un fenómeno de más de 100.000 km, y acá el evento cae a una mediana de ~12.800 km de odómetro (entrega v2). Antes de modelar verificamos si había una carga irreversible medible, que es la premisa de un modelo de degradación como el proceso gamma, y no la encontramos (`docs/memoria/f3-proceso-gamma.md`). El evento que vemos es una saturación por uso, recuperable, y no un desgaste acumulado. Eso orientó todo lo que sigue hacia los hábitos de manejo y el estado reciente del filtro.

1.2 Contexto: la experiencia del cliente

Qué entendimos. Ford pone la experiencia del cliente en el centro: eficiencia, consumo y comodidad. Para el conductor, una falla del filtro es un testigo que nadie le explicó, un auto que pierde fuerza y una visita al taller que no tenía planeada. Lo que más pesa en la relación con la marca es la sensación de que nadie le avisó.

Qué hicimos con eso. Diseñamos el producto alrededor de la confianza, no de la falla: el primer contacto es un consejo de manejo, en lenguaje humano, y no un turno en el taller. Al cliente nunca le llegan porcentajes de riesgo, códigos de error ni frases del tipo “tu auto va a fallar” (§5.2.1).

1.3 El desafío: predecir la degradación debida al uso

Qué entendimos. El documento habla de degradación “debido al uso indebido del cliente”: trayectos predominantemente cortos, operación que no completa los ciclos de recuperación del sistema y conducción urbana intensiva.

Qué encontramos en los datos. Esos patrones están, pero no todos se sostienen cuando se compara bien. Comparando fallados con sanos *del mismo mercado* y del mismo tramo de odómetro (§4.8):

- lo que crece hacia el evento es estar detenido con el motor en marcha (viajes de 0 km, el 33 % de las filas de `TripSummary`) y no llegar a temperatura;
- la velocidad y la fracción de viajes cortos entre los que se mueven *no* distinguen dentro del mercado. Con la primera entrega, la velocidad parecía la señal más fuerte, pero era el contraste entre Colombia (más lenta, falla más) y Chile.

Qué hicimos. Un modelo de deep learning que lee la secuencia de los últimos 1.000 km del auto (viajes y señales del filtro) y una capa de producto que traduce cada alerta en una acción, con IA generativa para redactar y código para verificar. Al conductor no le hablamos de “uso indebido”: le hablamos de hábitos que puede cambiar, comparados con autos sanos de su mercado.

2 Contexto del proceso

2.1 El proceso actual: la detección es reactiva

Qué entendimos. El vehículo tiene sensores que reportan por telemetría a una base en la nube, con triggers que arman y envían los paquetes. Los datos se generan de forma continua durante el uso normal. Hoy la degradación se identifica una vez manifestada: en un service o cuando el cliente la reporta.

Qué encontramos en los datos. El proceso reactivo está escrito en la telemetría, y eso es un riesgo para cualquier modelo:

- **La fecha registrada llega después del taller.** En la primera entrega, la fecha del evento coincidía con la intervención: el cambio de aceite se concentraba en ± 3 días, unas cinco veces la base. En la entrega v2, la fecha de los mismos autos cae siempre después, con una mediana de 14 días. En las dos semanas previas a la fecha nueva ya aparecen el cambio de aceite y el ralentí largo (≥ 15 minutos a 0 km, la regeneración forzada con el auto detenido), con 1,5–1,8 veces la base (`f9-entrega-v2.md` §6, `f9-eda-v2.md` §H).
- **Después del evento el uso cambia.** Después de la fecha, la fracción de viajes bajo régimen térmico baja en el 72 % de los autos y la velocidad sube en el 79 %.

Qué hicimos. Si el modelo viera el taller o lo posterior al evento, estaría haciendo exactamente la detección reactiva que Ford ya tiene. Por eso:

1. la referencia del evento es la **fecha registrada menos 21 días**, entre la mediana y el p75 del corrimiento medido;
2. el modelo nunca ve los **500 km previos** a esa referencia (el gap de blanking, §3.1.1);
3. nada posterior al evento entra a ninguna ventana, ni como positivo ni como negativo.

2.2 Ubicación en el flujo: posventa

Qué entendimos. El desafío se ubica en la posventa, con el vehículo ya en manos del cliente y a partir de la telemetría remota. La detección temprana impacta en el servicio (mantenimiento predictivo) y en la percepción de calidad.

Qué hicimos. Una alerta en un servidor no evita ninguna falla. El producto la convierte en una acción dentro del circuito de posventa (figura 1):

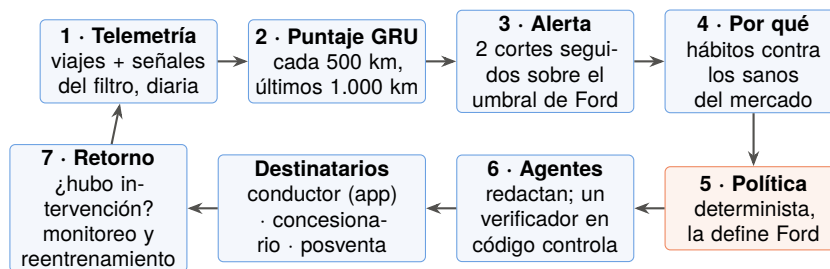


Figura 1: El circuito de posventa propuesto, de la telemetría a la acción. El modelo decide, el agente comunica y el código cuida lo que se dice.

Fuente: docs/pitch/presentacion-contenido.md bloque 4; feat/demo-gru:PRODUCT.md.

- **Con hábitos para nombrar**, el conductor recibe un consejo en la app (“comparado con autos sanos de tu zona, tu camioneta hace muchos viajes cortos; un tramo de ruta de 20 minutos por semana ayuda a que el filtro se limpie solo”).
- **Sin hábitos para nombrar**, lo contacta el concesionario con un resumen técnico.
- **Si el riesgo sigue alto dos revisiones después** (una revisión es un corte, cada 500 km), se escala al concesionario.
- **El gerente de posventa** recibe cada lunes una bandeja priorizada con todo redactado, lista para aprobar o editar.

El detalle del producto, la demo y sus números están en §5.2.1.

3 Definición del desafío

3.1 Desafío específico: anticipar el momento, sin hacer trampa

Qué entendimos. Ford pide un modelo que anticipe, a partir de patrones históricos de uso y variables de motor, *el momento en que un vehículo se aproxima a un evento de degradación*, a tiempo para una acción preventiva antes de que el cliente perciba el problema.

Esa frase tiene dos exigencias que tiran para lados distintos. *Anticipar* pide no mirar los últimos kilómetros antes de la falla, que es donde está la señal más fuerte. *El momento* pide saber *cuándo*, y no solo *qué auto*. La mayor parte del trabajo metodológico fue convertir esas dos exigencias en una pregunta medible que no se pudiera ganar con atajos.

3.1.1 La pregunta: un panel de cortes con gap de blanking

Armamos un panel con una fila por par (vehículo, punto de corte). Los cortes caen cada $\Delta = 500$ km de odómetro. En cada corte, el modelo ve solo los últimos $W = 1.000$ km y responde: ¿el evento cae entre $G = 500$ y $G + H = 3.500$ km después del corte? (figura 2).

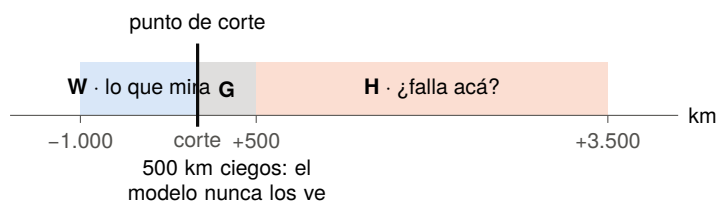


Figura 2: El planteo: ventana W hacia atrás, gap G y horizonte H hacia adelante. La etiqueta vale 1 si el evento cae en el horizonte.

Fuente: configs/data/panel_v2.yaml; docs/memoria/decisiones.md (18-09).

- **G = 500 km** es la regla antirreactiva. Sin ese gap, un modelo que mire los últimos kilómetros antes del evento memoriza los mensajes de “filtro sobrecargado” y reporta un número alto haciendo detección reactiva.
- **W = 1.000 km.** Con la primera entrega, cada 500 km más de ventana costaba unos tres eventos con positivo, y ventanas de 1.000, 2.000 y 3.000 km daban la misma métrica dentro del ruido.
- **H = 3.000 km** pone los positivos entre 500 y 3.500 km antes del evento, donde el perfil alineado muestra que la señal existe (§4.8).
- **El eje es el odómetro** para cortar las ventanas, pero el evento se ordena por tiempo y no por km: la dispersión del logaritmo es 0,38 en edad contra 0,91 en odómetro. Quien maneja más llega al evento con más km, no antes. Por eso la anticipación se reporta en km y también en días aproximados.
- **Sanos emparejados por odómetro y por mes.** Los eventos se concentran en ciertos meses. Sin emparejar por mes, sumar tres columnas de calendario subía el ROC de 0,67 a 0,76; emparejando, de 0,57 a 0,60 (primera entrega). El calendario no es el auto.

Con la entrega v2, el panel de desarrollo tiene 11.703 filas de 426 autos con cortes (135 fallados, 291 sanos) y 682 filas positivas: una tasa base de 0,058 por fila. Los autos sanos aportan en promedio 26,9 cortes y los fallados 28,8: las bolsas quedaron casi simétricas, algo que con la primera entrega no pasaba (9,0 contra 18,3) y que obligó a corregir las métricas (abajo).

3.1.2 Las reglas que no se negocian

1. **Gap de blanking:** el modelo nunca ve los 500 km previos al evento.
2. **Split agrupado por vehículo:** un auto nunca está a la vez en entrenamiento y en validación. La lógica vive en un solo módulo (`src/eval/splits.py`). Los folds se estratifican por la etiqueta a nivel vehículo, cruzada con mercado y motor, y la CV se repite tres veces.
3. **Holdout congelado:** 111 autos sorteados una vez, antes de mirar ninguna feature contra la etiqueta. Todo lo que se mira es desarrollo; las filas de test no se tocan hasta tener el modelo elegido.
4. **Features solo hacia atrás:** nada posterior al corte. Escalado e imputación se ajustan con el train de cada fold, dentro del pipeline.
5. **Auditorías obligatorias** antes de aceptar un número (§3.1.3, Anexo B).
6. **Nada se hardcodea:** paths, semillas e hiperparámetros salen de un YAML versionado.

3.1.3 Cómo medimos, y por qué cambiamos de métrica

Empezamos midiendo como manda el libro: PR-AUC por fila. Los primeros modelos daban números que parecían buenos, hasta que nos preguntamos qué pasaría con un modelo que solo reconoce *qué auto* falla, sin saber nada del *cuándo*.

El techo de cohorte

Todas las filas positivas del panel están dentro de los cortes de autos que fallan. Un modelo que puntúa cada fila con “¿este auto es de los que fallan?”, sin ninguna información del *cuándo*, saca **PR-AUC 0,177 y lift 3,03×** en el dev v2 (0,263 y 2,10× con la primera entrega). La meta inicial del plan (lift de 1,6–2×) se alcanzaba sin anticipar nada. Ningún modelo superó ese techo: **un PR-AUC por fila de este panel mide sobre todo qué auto y muy poco cuándo.**

Fuente: docs/memoria/decisiones.md, cierre de F3 (20-09); `src/eval/metrics.py::cohort_ceiling`.

Tres mediciones independientes llegaron al mismo lugar con la primera entrega: colapsar el puntaje de cada auto a su promedio *mejoraba* la decisión por vehículo; la auditoría (a') del modelo de

control daba negativa en las tres repeticiones; y un nulo que permuta la etiqueta dentro de cada auto quedaba por encima del modelo. Desde entonces la pregunta que decide es una sola:

De los autos que van a fallar, ¿a cuántos les avisamos a tiempo si solo permitimos una cantidad fija de falsas alarmas entre los autos sanos?

- **Detección por auto** con umbral exacto: para cada presupuesto b (2 a 30 % de los sanos con alguna falsa alarma), el umbral es el menor que deja a lo sumo b de los sanos alertados. Una alerta son **dos cortes seguidos** sobre el umbral. El criterio primario es la detección media al 5, 10, 15 y 20 %.
- **Contra el azar, con el mismo historial**: el puntaje permutado entre filas, 200 veces. Con la primera entrega, las bolsas asimétricas “detectaban” 10 % al 5 % sin ninguna señal; toda métrica que dependa del largo del historial se compara contra un nulo que lo conserve.
- **Contra la celda mercado × motor**: la tasa de fallas de la celda del auto, calculada sin él. Tres celdas (Chile × ENG_2, Colombia × ENG_2 y Brasil × ENG_3) concentran 117 de los 135 fallados de dev. Esa regla, sin mirar un viaje, detecta 18 · 33 · 62 % al 5 · 10 · 20 % [dev].
- **Fuera de muestra**: el umbral se fija con los sanos de los otros folds y se mide la tasa de falsas alarmas que realmente produce.
- **Bootstrap pareado por vehículo** para comparar dos modelos, con 1.000 a 2.000 réplicas.
- **AUC dentro de mercado × motor**: solo compara pares de autos de la misma celda. La diferencia con el AUC agrupado es composición de la muestra.

Para elegir candidatos, además, cada comparación nueva gastaba un lugar de un *presupuesto de comparaciones*: con unos 12 eventos por fold, sumar candidatos sobre la marcha garantiza que “el mejor” sea ruido. Las variantes importantes se preregistraron por escrito antes de medirlas (`docs/memoria/*preregistro*`).

3.2 Impacto operativo

Qué entendimos. No anticipar estos eventos afecta la satisfacción del cliente, sube los costos de garantía y reparación, reduce la vida útil percibida y desordena la planificación de mantenimiento de la red de concesionarios.

Qué hicimos con cada punto.

- **Satisfacción**: un aviso a tiempo, en lenguaje humano y con algo concreto para hacer. Si el auto no iba a fallar, el conductor igual recibió un consejo que le sirve.
- **Garantía y reparación**: la alerta llega con una mediana de unos tres meses de margen [test], tiempo para cambiar un hábito o programar un turno en vez de una urgencia. Cuántas fallas se evitan no se puede medir con estos datos: es la métrica central del piloto (§5.2.1).
- **Planificación de concesionarios**: la bandeja semanal prioriza los casos, y la perilla deja que Ford elija cuántas alertas acepta según su capacidad y el costo de cada una.

4 Datos técnicos relevantes

Esta sección es la respuesta al criterio de *calidad del tratamiento de datos*. Contamos qué llegó, qué trampas tenía, qué descartamos y por qué, y cómo quedó trazado cada paso. Las dificultades son parte del resultado: varias cosas que creíamos saber con la primera entrega dejaron de ser ciertas con la segunda.

4.1 Las tres tablas: lo que prometía el anexo y lo que llegó

Ford describe tres tablas (anexo 7 del desafío): *Trip Summary* (estado del vehículo al inicio y al final de cada viaje), *DynamicInformation* (estados en cada envío de datos) y *Vehicle General Information* (datos estáticos). Llegaron como seis CSV, cada tabla partida en dos cohortes de muestreo: vehículos con evento registrado (*Failed*) y sin evento (*NotFailed*).

Tabla 2: Los archivos de las dos entregas. La telemetría de los sanos de la entrega v2 es byte a byte la de la entrega 1: todo lo nuevo está en la estática y en los archivos de fallados.

Tabla	Cohorte	Filas, entrega 1 (15-09)	Filas, entrega v2 (26-09)	¿Igual?
Estática	fallados	378	298	no
Estática	sanos	742	768	no
Viajes (<i>TripSummary</i>)	fallados	947.477	934.463	no
Viajes (<i>TripSummary</i>)	sanos	1.583.958	1.583.958	sí
Señales (<i>DynamicInformation</i>)	fallados	4.202.484	3.595.216	no
Señales (<i>DynamicInformation</i>)	sanos	6.415.333	6.415.333	sí

Fuente: docs/memoria/f9-entrega-v2.md §1 y f1-datos-reales.md; reproduce scripts/compare_deliveries.py.

Qué encontramos al contrastar contra el anexo (detalle en el Anexo E):

- ***TripSummary* trae 25 de las 40 columnas documentadas.** Faltan 19: GPS (4), elevación (2), presión de neumáticos (8), hollín de regeneración manual (2), autonomía (1) y el DPF por su nombre (2). Aparecen 4 que el anexo no menciona: *AirFilterStart/End* y *AirRegenerationStart/End*, que son el DPF con otro nombre (§1).
- ***DynamicInformation* y la estática cierran (7 y 7 columnas), con renombres cosméticos:** *Vin* → *VehicleCode*, *SalesCountryCd* → *SalesCountry_cd*. No hay VIN: el único identificador es *VehicleCode*, y el join entre las tres tablas cierra.
- **El evento no está marcado fila a fila:** sale del vehículo. *IdentificationDate* está en el 100 % de los fallados y es nula en el 100 % de los sanos. **La cohorte es la etiqueta**, y por eso nunca entra como variable.
- **Regenerations no es un booleano:** es el texto literal "Regeneration" que marca una fila. Declararla numérica la convertía en nula en silencio.
- Los seis archivos vienen con BOM: si no se leen como *utf-8-sig*, la primera columna sale con el BOM pegado al nombre y el join falla.

4.2 Equipos o sistemas existentes: el histórico por vehículo

Qué entendimos. El documento promete un histórico de recolección por vehículo, con autos con y sin degradación, y deja la periodicidad y el volumen a confirmar.

Qué encontramos. Hay historial de sobra para ventanas de varios miles de km. Con la primera entrega, la mediana por vehículo era de ~2.000 viajes, ~7.000–8.000 señales, ~17.000–21.000 km de odómetro final y 400–470 días de historia. *TripSummary* tiene una fila por viaje y *DynamicInformation* una por mensaje de la ECU. Tres cosas del "sistema" importan más que el volumen:

- **Las dos cohortes se extrajeron en fechas distintas:** los viajes de fallados de la entrega v2 llegan al 24-09-2026 y los de sanos terminan el 14-09-2026. Hay 16.911 viajes de fallados posteriores al último de un sano. Un modelo que los viera aprendería "si hay datos después del 14-09, es un fallado".
- **Un contador se apagó:** el marcador *Regenerations* de *DynamicInformation* deja de registrar-

se el 25-05-2026 para toda la flota (cero marcadores de junio en adelante contra unas 2.000 caídas de nivel por mes en los viajes).

- **Las fechas mezclan dos formatos** en el mismo archivo (con y sin microsegundos). Sin declarar ISO 8601, pandas anula las del segundo formato en silencio: 4.464 inicios de viaje, 3.298 fines y 18.124 timestamps de señales. El crudo tiene cero nulos.

4.3 Restricciones técnicas: anonimización y renombres

Qué entendimos. Las variables vienen renombradas y, cuando corresponde, anonimizadas, para preservar la confidencialidad.

Qué encontramos y qué hicimos.

- Los vehículos son VEH_XXXX, los motores ENG_1 . . 3 y las series MODEL_x. Los países venían como CNTRY_1 . . 5; la entrega v2 trae el código ISO (ARG, BRA, CHL, COL, PER) y la ciudad de venta.
- El DPF llegó como “Air Filter” y “Acumulation”. Lo tratamos como *nivel de acumulación del filtro, escala 0–100*, que siempre es correcto, sin afirmar una semántica que Ford no confirmó.
- Sin GPS ni elevación por viaje, las features de altura previstas murieron. Con la entrega v2 probamos la altura de la ciudad de venta (geocodificada desde una fuente pública): va en la dirección de la física, pero no se separa del cero (§4.8). Sin presión de neumáticos tampoco hay features de esa familia.

4.4 Lo que encontramos: las trampas de los datos

Cada una de estas trampas rompe algo en silencio: no hay error, solo un número mejor de lo que debería.

Tabla 3: Las trampas que encontramos, cómo las vimos y qué hicimos.

Trampa	Cómo la vimos	Qué hicimos
Dos listas muestreadas en períodos de producción distintos	En la entrega v2, antes del 20-01-2025 solo hay fallados (93) y después de julio de 2025 solo hay sanos (341), contra unos 50 eventos esperados por su exposición. Sumando esos autos, la fecha de producción sola separa fallados de sanos dentro del mercado con AUC 0,87	Universo = período común, 20-01 a 31-07-2025 (§4.6)
La fecha registrada es posterior al taller	Cambio de aceite y ralentí largo (1,5–1,8× la base) en las dos semanas previas a la fecha v2; la fecha v1 de los mismos autos cae una mediana de 14 días antes	Referencia del evento = fecha – 21 días
El contador de regeneraciones se apaga	Cero marcadores desde el 25-05-2026 en toda la flota. Su correlación con la etiqueta (0,317) era calendario: contadas desde las caídas del nivel, cae a 0,072	Fuera del modelo (<code>aux_</code>); regeneraciones = caídas de <code>AirRegeneration</code> de al menos 15 puntos
13 autos con dos códigos	Mismos viajes bajo dos identificadores consecutivos (Jaccard 1,0 sobre fecha y odómetro de cada viaje); en la entrega 1, además, cada código aparecía en las dos cohortes	Se colapsan a 13 vehículos antes de cualquier split, con la etiqueta a favor del fallado
Formato de la entrega v2	Columna del evento renombrada solo en fallados; <code>ProductionDay</code> de fallados contado desde otro origen (+538 días exactos en los 15 códigos comunes); una fila por evento (19 autos con 2 eventos, uno con 4); 26 filas repetidas en sanos; extracción de fallados diez días más larga	Corrección al leer, declarada en <code>raw_sources.yaml</code> (<code>rename</code> , <code>offsets</code> , <code>truncate_after</code>) y probada en <code>check_setup.py</code>
La tasa por país depende de cómo se armó la lista	Va de 0 % (PER) a 48 % (COL), en el orden inverso de cuántos fallados sin fecha sacó Ford de cada mercado	Toda comparación fallado/sano, dentro del mercado; todo AUC, también dentro de mercado × motor
ENG_3 nunca fallaba (entrega 1)	0 fallados entre 268 ENG_3: muestreo, no física. En la v2 tiene 47 eventos, 46 en Brasil	Fuera del modelo con la entrega 1; vuelve con la v2, reportando siempre dentro de la celda
Después del evento el uso cambia	Bajo régimen baja en el 72 % de los autos y la velocidad sube en el 79 %	Nada posterior al evento entra a ninguna ventana

Fuente: `f9-entrega-v2.md`, `f9-universo-v2.md`, `f9-eda-v2.md`, `f1-clones-vehiculos.md`, `f1-sesgo-eng3.md`, `f2-eda-revision-y-features.md` §2.1.

Una pregunta que solo Ford puede contestar. Entre la entrega 1 y la v2 desaparecieron 285 códigos de fallados. 278 tenían la fecha por defecto (`IdentificationDate` igual a los días hasta la venta): Ford no corrigió esas fechas, sacó a esos autos. No sabemos si eran fallas reales sin fecha o un error de la consulta. Importa porque se concentran por mercado, y explican en parte por qué la tasa de fallas cambia tanto de un país a otro.

4.5 Limpieza, faltantes y atípicos

Tabla 4: Valores imposibles, faltantes y columnas con trampa, y cómo se trataron.

Problema	Magnitud	Tratamiento
Velocidad imposible	hasta 135.600 km/h (0,03 % de los viajes sobre 200 km/h)	Velocidad recalculada como km/duración, con tope físico de 200 km/h
KilometerPerHour nulo	~33 % en v2, <i>exactamente</i> en los viajes de 0 km (100 % de esas filas, 0,05 % del resto); donde existe, es km/duración con Pearson 0,99999	No se imputa: esas filas son ralenti y se cuentan aparte
Viajes de 0 km (ralenti)	33,4 % de las filas de <i>TripSummary</i> (v2)	Toda fracción “de viaje” se calcula entre los que se mueven; el ralenti es una señal propia, la que más anticipa
Temperaturas y duraciones extremas	-73 °C de ambiente, viajes de 20 días, viajes de -230 km (< 0,05 % cada uno)	Topes físicos fijos, no cuantiles (los cuantiles dependerían del train de cada fold): motor y refrigerante [-40, 130] °C, ambiente [-40, 60] °C, duración [0, 1.440] min
Nivel de combustible	supera 100 en el 10,5 % de los viajes (hasta 103,5)	No se recorta a [0, 100]: se trata como índice relativo
Vida del aceite	igual al inicio y al final del viaje en el 100 % de los casos	La feature es la pendiente del nivel contra el odómetro en la ventana, no la suma de deltas
Odómetro de señales nulo	11,15 % en la entrega 1, concentrado en 9 vehículos (mediana por vehículo 0,00 %); 0,5 % en los fallados v2	La ventana se ubica por el odómetro de los viajes, que está prácticamente completo (8 filas nulas)
Retrocesos del odómetro	165 viajes con km negativos en 24 vehículos (entrega 1)	Esos viajes y los que no tienen odómetro se descartan: no tienen una posición confiable en el eje de km
Umbral de regeneración	con caídas de 5 puntos, el saldo de caídas no sigue la acumulación medida (correlación 0,005)	Se exige una caída de al menos 15 puntos (correlación mediana 0,45 dentro del auto)
Escala de acumulación	llega a 100 (0,58 % de los fines de viaje); 95 es un escalón con masa, no el tope	Saturado = nivel ≥ 95
Bins sin viajes (secuencia)	tramos de 50 km sin registros	Un canal de cobertura (cantidad de viajes y de registros) más arrastre del último valor dentro de la misma ventana: un tramo vacío no es “cero ralenti”

Fuente: f2-calidad-columnas-dev.md, f1-calidad-odometro.md, f2-umbral-regeneraciones.md, f2-eda-revision-y-features.md §2, configs/data/features_v1.yaml (clip, thresholds), configs/data/panel_seq_trips_v2_estaticas.yaml.

Faltantes que no se imputan. Dos faltantes son estructurales y los tratamos como información: la velocidad nula en los viajes de 0 km (es ralenti, no un dato perdido) y la fecha del evento nula en los sanos (es la etiqueta, no una covariable). El resto de la imputación (mediana) y el escalado se ajustan con el train de cada fold, dentro del Pipeline, nunca sobre el panel entero.

4.6 El universo del estudio y el holdout

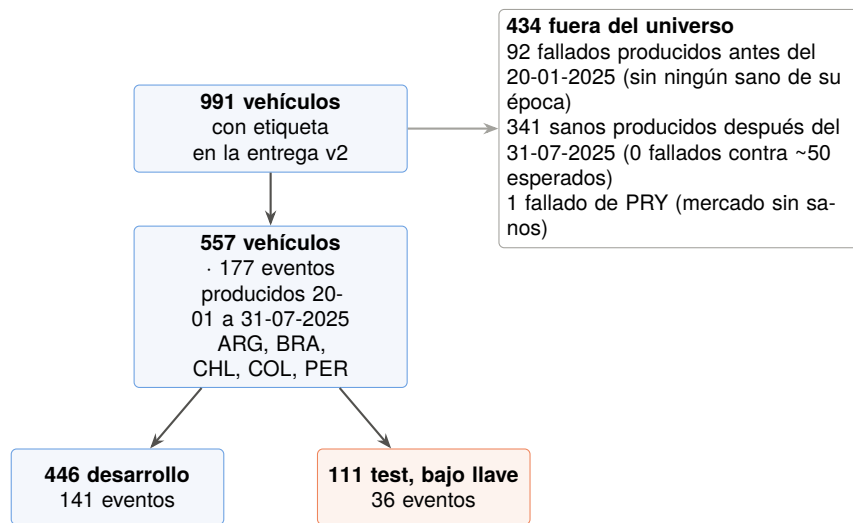


Figura 3: Del dataset al universo y al holdout. El holdout se sorteó una vez (semilla 42), estratificado por evento × mercado × motor, antes de mirar ninguna feature contra la etiqueta.

Fuente: docs/memoria/f9-universo-v2.md §3-§4; configs/data/test_split.yaml; reproduce scripts/make_test_split.py.

Por qué el período de producción. Es la misma regla simétrica que usamos con la primera entrega, aplicada en otro eje: se conserva el período donde las dos cohortes se muestrearon. Un sano producido en octubre de 2025 no es un negativo verificable si en esa época no se registró ninguna falla. El equipo propuso sacar el recorte para no tirar 433 autos, y lo probamos de punta a punta (990 vehículos): con los autos de afuera, la fecha de producción sola separaba fallados de sanos con AUC 0,87 dentro del mercado, y los modelos ordenaban *peor* a los autos comparables (survival stacking, de 0,571 a 0,471 dentro de mercado × motor). Volvimos a la ventana. Cortar por km no evita el problema, porque lo sesgado es quién entra en la muestra, no el eje.

Lo que costó re-sortear. Con la entrega v2 el holdout se volvió a sortear con la misma semilla: el universo, las etiquetas y los estratos cambiaron enteros. 36 de los 111 autos del test nuevo estaban en el desarrollo de la entrega 1 (13 con evento) y se habían mirado durante el análisis. Por eso el test se reporta también sin ellos (§5.1.3).

Tabla 5: Qué cambió al rehacer todo con la entrega v2. Lo que no se sostuvo, se dice.

Con la entrega 1 (364 autos, 2 mercados, 80 eventos)	Con la entrega v2 (557 autos, 5 mercados, 177 eventos)
La velocidad baja antes de la falla: la señal más fuerte	Desaparece dentro del mercado: era Colombia (lenta, falla) contra Chile
No hay trayectoria: el exceso de ralentí no crece hacia el evento	Sí crece: 0,60 → 0,64 → 0,76
El registro de eventos tiene una ventana de calendario (sep-2025 a mar-2026)	No hay ventana: era de la extracción vieja (el mes calendario no agrega nada, $p = 0,16$)
El riesgo crece con la edad sin parar	Sube hasta ~4 meses desde la venta (0,34 → 2,11 → 3,37 eventos por 100 autos-mes) y después queda en ~3-4
ENG_3 nunca falla y queda afuera	ENG_3 falla (46 de sus 47 eventos en Brasil) y entra
La fecha del evento es la intervención	La fecha cae ~2 semanas después: se corre 21 días
Rasgo temprano post-venta, AUC ~0,62-0,68	Se sostiene con 3× los eventos y en 5 mercados (~0,60-0,64)
Finalista: survival stacking con la ventana del registro (K2), ~15-17 % al 5 %	K2 pierde su corrección (la ventana) y no aprueba (a'); los secuenciales pasan adelante

Fuente: f9-eda-v2.md §1; decisiones.md (26-09); f9-remedicion-v2.md.

4.7 Las features: qué entró, qué no y por qué

El modelo recibe la ventana de 1.000 km de dos formas, según la familia de modelo (diccionario completo en el Anexo D):

- **53 features agregadas** (modelos tabulares y de supervivencia), en cuatro familias: **A · térmica y trayectos cortos** (17: ralentí, viajes bajo régimen, temperaturas, arranques en frío, viajes encadenados, distancia y duración), **B · regeneración** (15: regeneraciones por 1.000 km contadas desde las caídas del nivel, distancia entre regeneraciones, nivel residual, saturación, avisos del filtro al fin del viaje), **C · uso** (6: velocidad recalculada, fracción urbana, viajes y km por día, reposo entre viajes) y **D · severidad** (13: tasas de mensajes del filtro por 1.000 km, vida del aceite y su pendiente, consumo), más 2 controles de cobertura de la ventana.
- **La secuencia** (modelos recurrentes): la ventana en 20 tramos de 50 km × 15 canales. 9 canales salen de las señales (registros, cobertura, acumulación media y máxima, cuánto sube, caídas por regeneración, y tres fracciones de avisos) y 6 de los viajes (cantidad, ralentí, bajo régimen entre los que se mueven, velocidad, km y duración por viaje). Más país, motor y serie como estáticas.

Tabla 6: Qué datos entraron al modelo y cuáles quedaron solo para auditar.

Dato	Decisión	Por qué
Viajes: ralentí, temperaturas, duración, nivel del filtro, regeneraciones reconstruidas	entra	Es el uso y el estado del filtro, siempre hacia atrás del corte
Mensajes del filtro	entra como tasas por 1.000 km	Severidad. Sin el gap serían casi la definición del evento: por eso el gap existe
Mercado, motor y serie	entran (v2)	Aportan la tasa de su celda y no orden entre autos: todo modelo que los usa se reporta también dentro de mercado × motor
Fecha de producción, de venta, días hasta la venta	solo auditoría (aux_)	Miden la cohorte de muestreo, no el auto
Temperatura ambiente	solo auditoría	Deriva con el mes y con el lugar: los fallados viven en lugares más fríos que los sanos de su mercado
Marcador Regenerations y DistanceBetweenRegenerations	solo auditoría	Se cortan el 25-05-2026: miden el calendario
Interrupciones de regeneración	no se construye	1 mensaje de cada 100.000
Altura de la ciudad de venta	no entra	En la dirección de la física (RR 1,30 por cada 1.000 m, IC95 [0,93; 1,81]) pero no significativa, y la ciudad de venta no es la de uso
Los 433 autos fuera del período común	fuera de toda comparación fallado/sano	Con ellos la fecha de producción domina (AUC 0,87)

Fuente: decisiones.md (18-09 y 26-09); f9-eda-v2.md §I; configs/data/features_v1.yaml.

4.8 Qué dicen los datos: el análisis exploratorio

Todo lo de esta sección es de desarrollo [dev] y compara fallados contra sanos *del mismo mercado*. En la entrega v2 una comparación agrupada mezcla mercado con señal: con cinco mercados de tasas tan distintas, la versión agrupada infla lo que en realidad es país (por ejemplo, la fracción de avisos de filtro anormal pasa de 0,52 dentro del mercado a 0,58 agrupada).

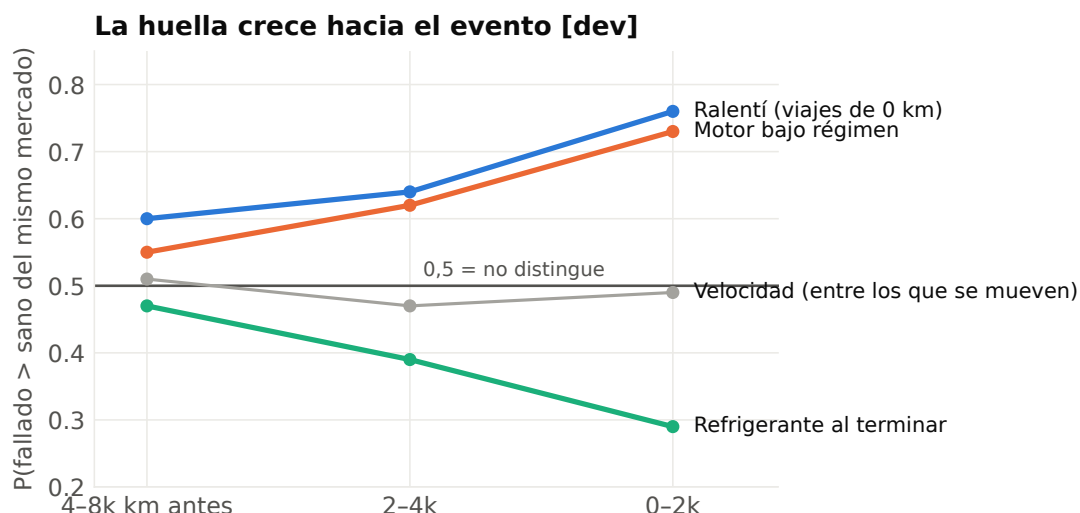


Figura 4: Probabilidad de que un auto que va a fallar supere a un sano del mismo mercado y del mismo tramo de odómetro, según la distancia al evento. Ralentí y bajo régimen crecen hacia el evento; el refrigerante termina los viajes más frío. La velocidad no distingue dentro del mercado.

Fuente: docs/memoria/f9-eda-v2.md §E (dev, 103–132 fallados por tramo); figura: scripts/make_report_figures.py.

- **La huella crece hacia el evento** (figura 4). Lo que crece es estar detenido con el motor en marcha y no llegar a temperatura, no manejar distinto. Con la referencia corrida tres semanas la tendencia se sostiene. Lejos del evento (4.000–8.000 km, unos 3–5 meses antes) la señal es débil.
- **Se ve temprano.** Con las features de los primeros 60 días después de la venta, el ralentí ordena autos con AUC 0,636 dentro del mercado (125 fallados, 298 sanos). Es señal de *qué auto*, disponible desde el primer o segundo mes.
- **Algunas cosas van al revés de lo esperado:** los que fallan terminan los viajes con el filtro levemente *menos* cargado (0,40–0,45) y regeneran algo menos por km (0,40–0,45). Por eso al conductor nunca se le nombra el estado del filtro (§5.2.6).
- **La temperatura ambiente de los fallados es más baja** que la de los sanos de su mercado (0,37–0,45), también contra el mismo mes: no es estación sino lugar. Una parte del “bajo régimen” puede ser clima de la ciudad. Por eso la temperatura ambiente no entra al modelo.

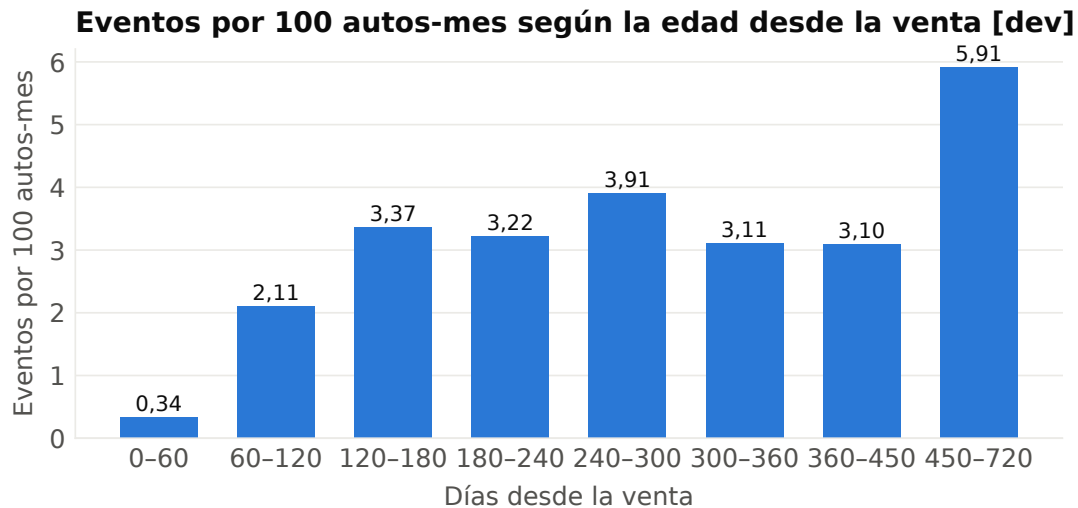


Figura 5: Riesgo por días desde la venta (Poisson con exposición, un registro por auto-día hasta el evento o el fin de extracción). Sube hasta los ~4 meses y entre los 4 y los 15 meses queda en ~3-4 eventos por 100 autos-mes.

Fuente: docs/memoria/f9-eda-v2.md §B (dev).

El riesgo es de edad, no de kilómetros (figura 5). Ajustando por días desde la venta, mercado y motor, el mes calendario no agrega nada ($p = 0,16$): no hay una ventana del registro. Lo que sí agrega es la *cohorte de venta*: los vendidos en el primer trimestre de 2025 fallan 2,1 veces más que los del segundo a igual edad ($p = 0,005$). Es el mismo gradiente de producción del universo, y por eso las fechas quedan fuera del modelo.

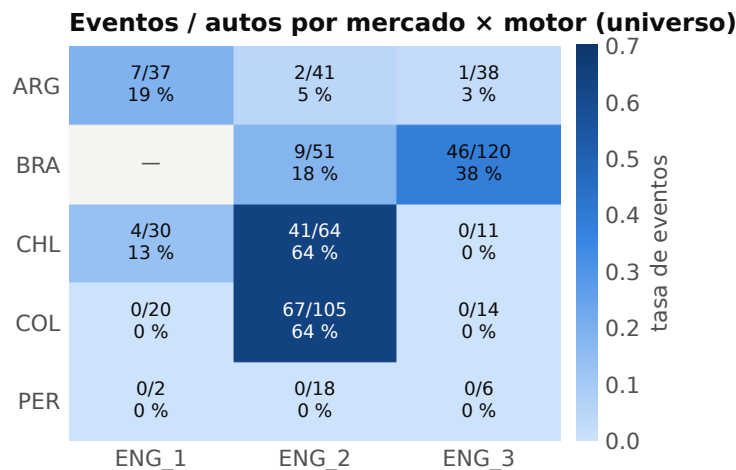


Figura 6: Eventos sobre autos por mercado × motor en el universo de 557. Tres celdas concentran casi todas las fallas. Las tasas por mercado están cruzadas con cómo Ford armó la lista de fallados, así que no se leen como física del mercado.

Fuente: docs/memoria/f9-universo-v2.md §3.

Dónde se concentra (figura 6). En el universo, Colombia × ENG_2 (67 de 105), Chile × ENG_2 (41 de 64) y Brasil × ENG_3 (46 de 120) concentran casi todas las fallas. Dentro de Brasil, ENG_3 falla 2,5 veces más que ENG_2 a igual edad, mientras que en Chile y Colombia ENG_1 y ENG_3 casi no fallan. Esto define el piso más exigente del proyecto: saber mercado y motor, sin mirar un solo viaje, ya ordena muchísimo. Qué parte de esa tasa es física (combustible, altura, motor) y qué parte es cómo se armó la lista no se puede separar con estos datos: es la pregunta más

importante para Ford.

4.9 Consistencia y trazabilidad del pipeline

- **Las correcciones de los crudos se declaran, no se programan sueltas.** El loader (`src/data/loader.py`) aplica por parte y por tabla lo que dice `configs/data/raw_sources.yaml`: renombres, corrimientos de origen, corte en el fin de extracción y formato de fechas. La entrega 1 se sigue leyendo con su propio YAML (`raw_sources_v1.yaml`), bit a bit.
- **Un solo lugar por decisión.** El universo vive en `src/data/usable.py`, los clones en `configs/data/vehicle_dedupe.yaml`, el split en `src/eval/splits.py`. El recorte a desarrollo se hace por pertenencia explícita a la lista congelada, y el código falla si el panel trae un vehículo excluido (señal de que se construyó con el universo viejo).
- **Holdouts congelados y reproducibles.** El de la entrega 1 se reproduce bit a bit (huella `ba9aa4d290bc6610`); el de la v2 guarda además qué autos estaban del otro lado antes.
- **Contrato de datos por prefijo.** Toda columna que entra a un modelo se llama `feat_` o `static_`; lo que sirve para auditar se llama `aux_` y el entrenamiento no lo puede ver. Agregar una feature es una línea de YAML.
- **Nada se hardcodea:** cada corrida es un YAML versionado en `configs/`, y sus métricas y su configuración completa quedan en `results/`. Los paneles y el holdout se publican como artefactos de Weights & Biases, en un único proyecto compartido.
- **Chequeos automáticos** (`scripts/check_setup.py`, más de 200) que cubren el contrato, las trampas de formato y las reglas antileakage; se corren antes de cada merge.
- **Cada hallazgo tiene su evidencia y su comando** en `docs/memoria/`, y cada decisión su fecha y su motivo en `docs/memoria/decisiones.md`. Los comandos para reproducir este informe están en el Anexo A.

5 Expectativas de solución

Ford espera que la solución tenga algún aspecto de por lo menos uno de cuatro campos: análisis de datos, inteligencia artificial, machine learning o deep learning. La nuestra usa los cuatro:

- **Análisis de datos:** las auditorías de los crudos, el universo y la comparación dentro del mercado (§4).
- **Machine learning:** modelos tabulares, de supervivencia en tiempo discreto y de cura, con validación agrupada y auditorías de leakage.
- **Deep learning:** el finalista es una red recurrente con atención sobre la secuencia de la ventana.
- **IA generativa:** agentes que redactan los avisos, controlados por un verificador en código.

5.1 Objetivo de mejora: anticipar con el mayor margen posible

5.1.1 El recorrido: más de 40 modelos, la misma cuenta

Probamos de lo simple a lo complejo, y cada familia nos enseñó algo. La tabla 7 resume el camino. Las cifras de la entrega 1 no se comparan con las de la v2: la población, las etiquetas y el nulo cambiaron.

Tabla 7: Qué probamos, qué dio y qué aprendimos. Todo es de desarrollo salvo donde dice [test].

Modelo o idea	Resultado	Qué aprendimos
Entrega 1 (290 autos de desarrollo, 2 mercados)		
Logística y LightGBM sobre 53 features	PR-AUC por debajo del techo de cohorte; (a') del control $-0,0055 \pm 0,0040$	Aprenden <i>qué auto</i> , no <i>cuándo</i> . Cambiamos la métrica (§3.1.3)
Proceso gamma de degradación	No se implementó	No hay carga irreversible medible a estos kilometrajes (el 51 % de las regeneraciones termina en 0 exacto)
Horizonte ordinal (bins)	No reemplaza al binario	Afinar bins no mueve la aguja
TimesFM zero-shot (Google)	No le gana a la tasa base	Sugirió medir el desvío respecto de la historia del auto; lo medimos y tampoco sumó
CNN-LSTM de la tutora	11,9 % $\pm$ 3,6 al 5 % (3 repeticiones)	Con pocos eventos, las redes no tienen con qué aprender
Survival stacking (hazard por bin de km)	Único con (a') positivo estable ($+0,016 \pm 0,003$); calibrado (Brier 0,112)	Finalista de F3: elegido por (a') y estabilidad, no por PR-AUC
Efecto aleatorio por auto (GPBoost)	Pierde contra el apilado simple	El efecto por auto absorbe justo lo que distingue autos
Cure model por hito post-venta	Señal real ($p = 0,005$) pero empata con un solo número de uso (km por día)	El rasgo temprano existe y es débil
K2: survival stacking con la ventana del registro	17,0 % $\pm$ 3,8 al 5 %; el embolsado, 15,6 %	Finalista de la entrega 1; el pitch de entonces citaba ~15–17 %
Entrega v2 (446 autos de desarrollo, 5 mercados, 135 fallados con cortes)		
K2 sin su ventana	Falla (a'); survival stacking detecta ~11–14 % al 5 %	La corrección de K2 era la ventana, que en v2 no existe
Re-medición de 42 modelos	GRU + viajes + estática $\times 3$: 27 · 42 · 61 % al 5 · 10 · 20 %	Con 3× los eventos, los secuenciales pasan adelante; ninguno le gana con evidencia a la celda
Sweep bayesiano de la GRU (130 trials)	El mejor empata con semillas nuevas (0,469 contra 0,471)	La semilla mueve más que los hiperparámetros (0,395–0,476): se reporta un ensamble de 3
GRU con etiqueta suave lejos del evento	+9,6 [3,0; 14,4] en dev; -13,3 [-25,0; +3,9] [test]	Ganaba en dev y no se sostuvo (§5.1.5)
10 semillas y ensambles heterogéneos	Ninguno se mueve más de ± 3 puntos al 5–10 %	Hay un techo de información, no de modelo
MiniRocket (kernels aleatorios + ridge)	16 · 27 · 39 % contra 28 · 43 · 61 % de la GRU; -16 al 10 % [-26; -8]	Pierde con evidencia: las convoluciones aprendidas usan mejor la poca señal

Fuente: decisiones.md (20-09 a 01-10); f3-mejor-modelo-a-la-fecha.md; f9-remedicion-completa-v2.md; f10-sweep-gru.md; f11-*.md; f12-preregistro-deteccion.md; MiniRocket: origin/exp/minirocket:docs/memoria/f13-preregistro-minirocket.md.

Tabla 8: Los modelos principales re-medidos sobre el dev v2 [dev]: detección (%) con umbral exacto, promedio de 3 repeticiones; AUC por auto agrupado y dentro de mercado $\times$ motor.

Modelo	AUC auto	AUC celda	5 %	10 %	20 %
GRU + viajes + estática $\times 3$ (finalista)	0,82	0,65	27	42	61
CNN-LSTM + viajes + estática $\times 3$	0,81	0,62	22	41	62
CNN-LSTM de la tutora + estática $\times 3$	0,82	0,62	27	40	68
Survival stacking (F3)	0,72	0,57	14	24	42
LightGBM (control, 53 features)	0,72	0,57	18	28	42
Logística L2	0,71	0,58	15	23	37
Celda mercado $\times$ motor (sin modelo)	0,81	0,46	18	33	62
Solo el odómetro del corte	0,42	0,43	3	6	15
Azar (puntaje permutado, historial conservado)	—	—	5	11	21

Fuente: docs/memoria/f9-remedicion-completa-v2.md (42 modelos, con la tabla completa y las comparaciones pareadas).

semillas son las del reporte v2 y no se eligieron por resultado. La alerta salta con dos cortes seguidos sobre el umbral.

- **Nadie le explicó el filtro.** La relación entre viajes cortos y hollín la encontró en los datos (§5.2.6).

5.1.3 El resultado, en autos que nunca vio

El test son los 111 autos del holdout. 103 tienen cortes en el panel: 32 fallaron y 71 no. Cada modelo se entrenó con todo el desarrollo (446 autos) y se midió una vez con la misma cuenta que en dev (`scripts/eval_test.py`).

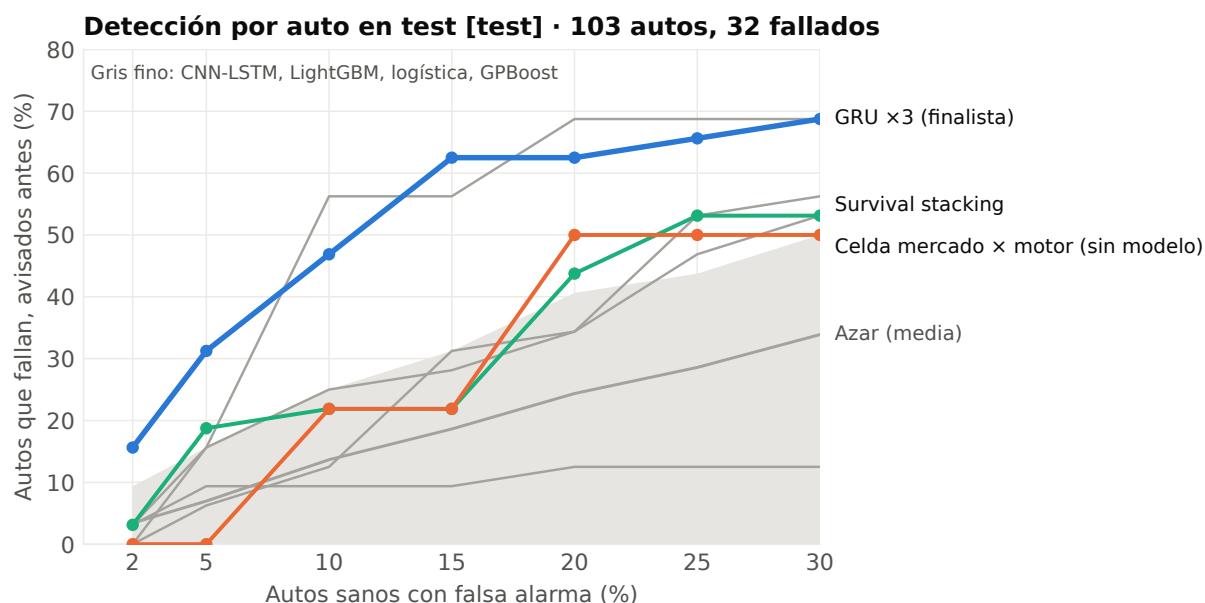


Figura 8: Detección de autos que fallan según el porcentaje de sanos con falsa alarma, en test. La banda gris es el azar hasta su percentil 95. La celda mercado x motor no detecta nada por debajo del 10 % porque alerta celdas enteras.

Fuente: `experiments/test-modelos-v2/curve.csv` (`configs/eval_test_modelos_v2.yaml`); figura: `scripts/make_report_figures.py`.

Tabla 9: El finalista en test [test]: detección (%) por presupuesto de falsas alarmas.

	5 %	10 %	15 %	20 %	Primario (5–20 %)
GRU x3 (semillas 42, 1, 2)	31,2	46,9	62,5	62,5	50,8
Survival stacking (finalista de la entrega 1)	18,8	21,9	21,9	43,8	26,6
Celda mercado x motor (sin modelo)	0,0	21,9	21,9	50,0	23,4
Azar, media	7,0	13,7	18,6	24,4	—
Azar, p95	15,6	25,0	31,2	40,6	—

Fuente: `experiments/test-f10-s42/report.json`, `experiments/test-modelos-v2/report.json`; docs/ memoria/f11-test-resultado.md. Todos los modelos, en el Anexo C.

- **10 de 32 al 5 %:** casi uno de cada tres, 4,5 veces el azar y por encima de su p95. Al 20 %, seis de cada diez.
- **Le duplica el primario a los tabulares y a survival stacking** (50,8 contra 21–27). Contra survival stacking, +24,2 puntos, IC95 [0,8; 43,0]: el intervalo apenas no cruza el cero, y la comparación no estaba preregistrada.

- **Contra la celda mercado × motor: +27,3 puntos, IC95 [-0,8; 46,1],** $p(\text{diferencia} \leq 0) = 0,028$. Queda en el límite y *no* lo citamos como “le gana”.
- **Ordena autos:** AUC por auto 0,794; dentro de mercado × motor, 0,675. Entre autos del mismo país y motor, el orden es moderado.
- **Repitió lo de dev:** en validación cruzada daba 27,4 · 42,5 · 53,3 · 60,7 % (primario 46,0).

Tabla 10: El punto de operación con el umbral fijado en dev [\[test\]](#): se elige el umbral con los sanos de desarrollo (15 modelos de fold) y se mide en test cuántas fallas detecta y cuántas falsas alarmas produce de verdad.

Modelo	Detección (%) al 5 · 10 · 20 %	Falsas alarmas reales (%)
GRU ×3 (finalista)	20 · 34 · 57	5,8 · 8,9 · 18,2
CNN-LSTM ×3	16 · 34 · 55	5,6 · 8,4 · 14,8
Survival stacking	13 · 20 · 33	4,6 · 8,3 · 15,6
LightGBM (53 features)	16 · 25 · 36	7,5 · 10,7 · 16,9
Logística L2 (53 features)	12 · 16 · 33	8,8 · 11,8 · 18,3
SS + efecto aleatorio (GPBoost)	8 · 11 · 14	6,5 · 11,3 · 21,5

Fuente: experiments/test-modelos-v2/operating.csv.

El número para operar. En producción el umbral no se puede elegir mirando los autos que se quieren puntuar. Fijado con los sanos de desarrollo, la GRU detecta 20 · 34 · 57 % al 5 · 10 · 20 % con **5,8 · 8,9 · 18,2 % de falsas alarmas reales**: el umbral cumple su presupuesto en autos nuevos (tabla 10).

Por qué citamos un rango y no un número

La configuración de la GRU se leyó tres veces en test: con las semillas 42/1/2 el 27-09 (34 · 41 · 63 % al 5 · 10 · 20 %), con las semillas 101-103 como referencia del tiro de F11 (28 · 50 · 50 %), y con 42/1/2 el 01-10 (31 · 47 · 63 %). La primera lectura fue antes del preregistro de F11 y no pudo inflar ese resultado, pero el test ya se había mirado. Además, la GRU no es reproducible bit a bit entre corridas, aun con la misma semilla (correlación 0,88-0,91 entre los puntajes de dos corridas con la misma configuración). Con 32 fallas, cada auto mueve 3 puntos. Por eso: **~30 % al 5 %, ~40-50 % al 10 % y ~50-60 % al 20 %**.

Sin los 36 autos de test que estaban en el desarrollo de la entrega 1 (71 autos, 22 fallados): 36,4 · 50,0 · 50,0 · 59,1 % al 5 · 10 · 15 · 20 % (primario 48,9). Lo aprendido mirando la entrega 1 no está inflando el test.

Fuente: docs/memoria/f11-test-resultado.md §2 y resumen.

5.1.4 Cuánto antes: el margen medido

Este es el objetivo de mejora de la sección 5.1 del desafío: anticipar con la mayor cantidad de tiempo o kilometraje posible.

- **Al 10 % de falsas alarmas [\[test\]](#),** la primera alerta llega con una mediana de **4.646 km** (p25-p75: 2.403-9.263 km) antes del evento, y unos **99 días** aproximados con los km por día de cada auto. Al 5 %: 3.987 km, ~72 días. Al 20 %: 5.969 km, ~109 días.
- **En dev [\[dev\]](#),** al 10 %: mediana ~7.600 km (p25-p75: 3.700-13.100), ~86 días. En km el test da menos porque sus autos detectados andan menos km por día; en días es parecido. Sin los autos vistos en la entrega 1, la mediana en test es de ~8.800 km.
- **El piso es el gap:** por construcción, ninguna alerta usa los 500 km previos al evento. La distancia se mide hasta la referencia del evento (fecha registrada - 21 días); hasta la fecha registrada, el margen es algo mayor.

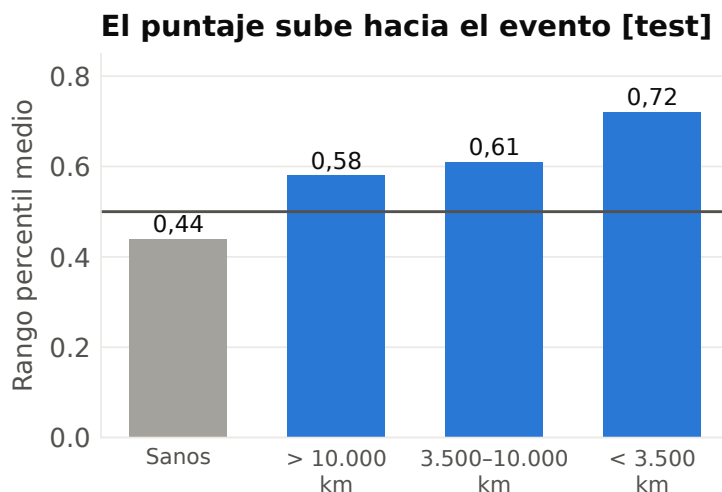


Figura 9: Rango percentil medio del puntaje en los cortes de autos que fallan, según la distancia al evento, y en los sanos. El fallado ya está por encima de los sanos lejos del evento (*qué auto*) y sube al acercarse (algo de *cuándo*).

Fuente: docs/memoria/f11-test-resultado.md §4 (test, promedio de las 3 semillas).

Cómo se lee el margen. El puntaje sube a medida que se acerca el evento (figura 9), y la primera alerta cae a mitad del historial, no en el primer corte: hay algo de *cuándo*. Pero la dispersión es enorme, y la auditoría (a') del finalista da +0,001 (§5.1.6). Por eso la frase defendible es “*lo marcamos con unos tres meses de margen*”, nunca “*predecimos que falla en tres meses*”.

5.1.5 Una variante que ganaba en desarrollo y no se sostuvo

Es el aprendizaje metodológico más importante del proyecto. Con la etiqueta dura, un corte a 10.000 km del evento de un auto que falla se entrena igual que uno de un auto sano, aunque la detección se cuente por auto. La variante F11 entrenaba esos cortes lejanos con 0,15 en vez de 0 (una etiqueta suave).

- **En desarrollo ganaba con evidencia**, en una confirmación preregistrada con folds y semillas nuevos: +9,6 puntos de primario, IC95 [3,0; 14,4], $p = 0,0015$ (34,6 · 48,6 · 60,2 · 70,1 % al 5 · 10 · 15 · 20 %). Era el primer modelo que le ganaba con evidencia a la celda.
- **En el tiro preregistrado en test no se sostuvo**: 31,2 contra 44,5 de la GRU con la etiqueta dura, -13,3 puntos, IC95 [-25,0; +3,9]. La regla escrita antes decía “no se sostiene”, y así se reportó, sin buscar variantes.
- **Por qué**: en test se comportó como la tasa de la celda (al 10 y al 20 % detecta exactamente lo mismo que la celda sola) y perdió el orden dentro de la celda (AUC 0,689 en dev, 0,571 en test). El 0,15 en todos los cortes lejanos premiaba reconocer a *ese auto* en cualquier momento, y lo más estable de un auto es su país, su motor y su serie.
- **Lo que nos llevamos**: la confirmación usó folds y semillas nuevos, pero los mismos 446 autos sobre los que se habían explorado unas 20 variantes. Un holdout que nadie toca es lo único que separa una mejora real del optimismo de desarrollo. El finalista es la GRU con la etiqueta dura porque era la referencia preregistrada y el candidato no se sostuvo, no porque se haya elegido entre varios modelos mirando test.

5.1.6 Límites

Lo que el modelo no puede afirmar

1. **No le gana con evidencia a la celda mercado × motor [test]** (+27,3, IC95 [−0,8; 46,1]). Parte de lo que detecta es composición: el país pesa (§5.2.6).
2. **Sabe poco del cuándo:** (a') = +0,001 [dev] (colapsar el puntaje al promedio del auto casi no cambia nada). Es positivo, como en todos los modelos de la v2, pero chico.
3. **Con 32 fallas, los intervalos son anchos**, y el orden entre familias tampoco es firme: sin los autos vistos, LightGBM y survival stacking suben a ~50 de primario y empatan con la GRU (survival stacking – GRU: +1,1, IC95 [−20,5; +27,3]).
4. **El test se leyó más de una vez:** se cita un rango. Ninguna variante nueva se mide contra él; una mejora sobre este modelo necesita datos nuevos.
5. **La muestra está enriquecida en fallas** (135 de 426 autos de desarrollo con cortes; 32 de 103 en test). Los conteos de alertas no se trasladan a una flota real, y el puntaje ordena autos: no es una probabilidad calibrada a la prevalencia real.
6. **No predecimos la fecha exacta:** marcamos con margen, con una dispersión grande.
7. **La referencia del evento corrida 21 días es poblacional:** no se puede fechar la intervención auto por auto.

5.2 Criterios de evaluación

5.2.1 Viabilidad técnica

Lo que ya existe y funciona.

- **Un modelo medido en autos que nunca vio**, con su umbral de operación fijado fuera de muestra y cumpliendo el presupuesto de falsas alarmas (§5.1.3).
- **Un pipeline reproducible** de punta a punta, desde los seis CSV hasta el puntaje, con las correcciones declaradas y más de 200 chequeos automáticos (§4.9).
- **Una demo de producto** desplegada (§5.2.1) que reproduce la flota de desarrollo semana a semana.

Por qué escala sin hardware nuevo.

- **Cero sensores nuevos:** solo resúmenes por viaje y mensajes del filtro que el auto conectado ya manda.
- **Modelo liviano:** ~3.900 parámetros, en CPU, sin GPU ni para servir ni para reentrenar. Hoy las 56 corridas de la re-medición completa tardaron ~25 minutos en 24 núcleos.
- **El mismo código de features en entrenamiento y en producción** (`src/features`, `src/data/panel.py`). Si producción reimplementara las features, los números del piloto dejarían de ser los medidos.
- **Los agentes trabajan solo sobre las alertas**, no sobre la flota: el costo de IA crece con los avisos, no con los autos.

Para un millón de autos con ~3 viajes por día, estimamos un batch diario de 1–2 horas en 8–16 vCPU (~3 millones de filas de viajes por día). Es una estimación propia que hay que validar con el equipo de datos de Ford.

Tabla 11: Cómo llega a producción.

Etapa	Qué	Duración	Criterio para pasar
0 · Congelar	Reentrenar con los 557 autos y empaquetar el modelo (pesos por semilla, export ONNX, referencia del ensamble, umbrales, 20 autos de control)	1–2 semanas	El paquete re-puntúa los autos de control igual
1 · Piloto en sombra	Puntuar la flota real de un mercado (Colombia o Chile, los de más eventos) sin avisar a nadie	3–6 meses	Tasa de alertas $\approx$ presupuesto elegido; detección realizada $\geq$ celda mercado $\times$ motor
2 · Piloto activo	Avisos en una región, con un grupo control sin aviso	6 meses	Fallas evitadas contra el control; costo por alerta; cambio de hábitos
3 · Escala	Todos los mercados validados, umbral recalibrado por mercado	continuo	Monitoreo en verde

Fuente: docs/pitch/presentacion-contenido.md bloque 5.

Por qué la sombra es el paso clave. Todo lo que medimos salió de las listas que Ford armó para el desafío, muestreadas en dos cohortes. La sombra es la primera medición de la prevalencia real por mercado $\times$ motor y de las falsas alarmas sobre la flota completa. También responde la duda más grande del proyecto: qué parte de lo que el modelo aprende es física y qué parte es cómo se armaron las listas.

Monitoreo y reentrenamiento. Tasa de alertas por mercado contra el presupuesto; deriva de las features por mercado y mes; detección realizada (cada falla en taller, ¿tuvo alerta antes y con cuánto margen?); datos que dejan de llegar (ya pasó con el contador de regeneraciones); rechazos del verificador. Reentrenamiento trimestral o con +50 % de eventos nuevos, con las mismas auditorías y comparado en sombra contra el modelo vigente.

Lo que falta escribir: hoy el entrenamiento guarda predicciones, no modelos; el empaquetado (`export_model.py`) está diseñado pero no implementado. No toca nada de lo medido.

El producto, funcionando: la demo. La demo (Streamlit, desplegada con clave) reproduce semana a semana los 426 autos de desarrollo con cortes (135 que fallan y 291 sanos), del 03-03-2025 al 24-09-2026, con el modelo finalista tal cual. El test no se muestra nunca: el bundle falla si aparece un auto de test. Tiene tres pantallas:

- **Bandeja** (el gerente de posventa, cada lunes): una tarjeta por alerta con el mensaje al conductor, el resumen del taller y los hechos en los que se apoya cada texto, con el tilde del verificador.
- **Vehículo** (el concesionario): el puntaje contra el umbral, la alerta y los hábitos en los que el auto se aparta de los sanos de su mercado.
- **Qué pasó después:** el replay al lado de los números oficiales.

Una **perilla** arriba de cada pantalla elige el punto de operación (5, 10, 15 o 20 % de sanos con un aviso de más) y mueve todo lo demás.

Tabla 12: La demo por punto de la perilla [dev]. El replay es una repetición de la CV; los oficiales, el promedio de tres.

	5 %	10 %	15 %	20 %
Detección oficial (3 repeticiones)	28,1	42,5	53,1	61,0
Fuera de muestra: detección / falsas alarmas reales (%)	27,9 / 4,9	42,5 / 10,2	52,3 / 15,1	60,7 / 19,9
Azar, media (mismo historial)	5,4	10,7	15,7	20,7
Replay: fallas avisadas antes (de 135)	39	56	69	82
Replay: sanos con un aviso de más (de 291)	14	29	43	58
Replay: alertas / escalamientos	53 / 7	85 / 10	112 / 22	140 / 36
Replay: alertas con un hábito para nombrar	48	73	101	125
Replay: anticipación mediana, alerta → falla (semanas)	15	16	16	21
Replay: ídem, en km	7.000	7.200	7.500	8.100

Fuente: `feat/demo-gru:docs/memoria/f9-demo-gru-final.md`; `bundle experiments/demo-bundle-gru-final`.

Los agentes y el verificador. Dos agentes (`gpt-5.4-mini`): el *redactor* escribe el mensaje al conductor, el resumen para el taller y la línea de la bandeja, eligiendo recomendaciones de listas cerradas; el de *triage* recorre los eventos de la semana con herramientas, pide la acción a la política (no puede elegir otra) y arma el resumen semanal. El *verificador*, en código, rechaza todo número que no esté en los hechos, el lenguaje causal (“porque”, “debido a”), las certezas, las promesas de que el riesgo baja, la palabra “probabilidad”, los síntomas del filtro en un texto al conductor y la atribución (“el sistema lo marcó por...”). Si rechaza, el agente reintenta hasta dos veces; si no lo logra, queda una plantilla aprobada. En la temporada completa, con los cuatro puntos de la perilla, los agentes redactaron **465 textos y 196 resúmenes**, todos aprobados por el verificador, con 40 intentos rechazados en el camino y ninguna plantilla. Cada respuesta queda en una caché por el hash de lo que la define, así que la demo se reproduce sin red.

5.2.2 Costo

Lo que cuesta operar el producto. Estimación propia para un millón de autos conectados, a validar con Ford (tabla 13). El 80 % del costo anual es la persona que lo opera: agregar autos casi no lo mueve. El costo real no es la computadora, sino integrar el sistema con Ford (ingesta, app, concesionarios) y el costo de cada alerta.

Tabla 13: Costo anual de operación para 1 millón de autos (estimación propia).

Componente	USD por año	Supuesto
Features + puntaje diario	2.500–6.000	batch de 1–2 h en 8–16 vCPU
Agentes (LLM)	1.000–3.000	~65.000 alertas por año × ~6 llamadas, modelo chico
Reentrenamiento + monitoreo	5.000–10.000	trimestral, en CPU
Equipo de operación	60.000–80.000	1 persona de ML/MLOps
Total	~70.000–100.000	~USD 0,07–0,10 por auto y por año

Fuente: `docs/pitch/presentacion-contenido.md` bloque 6. Implementación, una sola vez: ~USD 70.000–110.000 (tres personas durante 6 meses, incluye la sombra).

Lo que no citamos: el costo de una falla. No conocemos lo que le cuesta a Ford. Con la primera entrega armamos escenarios con precios públicos de EE. UU. (diagnóstico USD 100–250, falla USD 800–15.000) sobre el finalista de entonces (`f8-costos-k2.md`). Lo que dejaron es una forma de pensar, no un ahorro:

- si la falla es barata, lo óptimo es no alertar; si la alerta es casi gratis, alertar a todos le gana a cualquier modelo;

- el modelo paga en una franja: cuando el beneficio de anticipar es de ~5 a ~15 veces el costo de una inspección y la prevalencia real es de 2–5 %;
- por eso pedimos a Ford dos números: cuánto le cuesta una falla contra un diagnóstico, y la prevalencia real.

Por qué el diseño baja el costo de equivocarse. El primer contacto es un consejo de manejo, no un turno: al 5 %, 48 de las 53 alertas del replay llevan un hábito para nombrar [dev]. Si el auto no iba a fallar, el conductor recibió un consejo que igual le hace bien al motor. Pero en una flota real la mayoría de los avisos van a ir a autos que no iban a fallar, porque la prevalencia es mucho más baja que en la muestra:

Tabla 14: Cuenta ilustrativa por cada 1.000 autos, con la detección de test (31 % al 5 %, 62 % al 20 %).

Prevalencia supuesta	Perilla	Fallas avisadas	Consejos de más	De cada 10 avisos, cuántos son de un auto que iba a fallar
2 %	5 %	6 de 20	49	~1
5 %	5 %	16 de 50	48	~2
10 %	5 %	31 de 100	45	~4
5 %	20 %	31 de 50	190	~1

Fuente: docs/pitch/presentacion-contenido.md bloque 6 (respaldo).

Es un argumento a favor del diseño: si el aviso de más fuera un turno en el taller, la fricción sería inaceptable. Y mandarle el mismo consejo a toda la flota no sirve, porque un aviso genérico se ignora: el nuestro dice algo cierto de ese auto (“comparado con los sanos de tu zona”). Con estos datos no pudimos medir la pérdida de consumo (el nivel de combustible solo permite estimarla en una parte de los viajes y no separa fallados de sanos): es la primera métrica del piloto.

5.2.3 Carácter innovador

1. **Una métrica que no se deja engañar.** Mostramos que en este problema el PR-AUC por fila se alcanza sin anticipar nada (el techo de cohorte), y lo reemplazamos por la detección por auto con un presupuesto de falsas alarmas, contra un nulo que conserva el largo del historial y contra la regla “país × motor”. Es la diferencia entre un modelo que reconoce autos y uno que avisa a tiempo.
2. **Una red que lee la ventana en orden.** La GRU con atención sobre tramos de km ve tendencias dentro de los 1.000 km que un agregado borra, y le duplica la detección a los modelos tabulares en test.
3. **IA generativa con barandas.** El modelo decide, el agente comunica y el código verifica. El LLM nunca produce un número, un puntaje ni un diagnóstico; un verificador rechaza lenguaje causal, certezas y números que no estén en los hechos.
4. **El porqué contado como comparación, no como causa.** Al conductor se le nombran solo hábitos que puede cambiar, en los que su auto se aparta de los sanos de su mercado y que coinciden con la física del DPF.
5. **La perilla.** Ford elige cuántas falsas alarmas tolera según lo que le cuesta cada una, con el umbral fijado fuera de muestra.
6. **Rigor como producto.** Preregistros escritos antes de medir, auditorías obligatorias y un holdout que se abrió una sola vez para elegir. La confianza que le pedimos al jurado es la misma que el producto construye con el cliente.

5.2.4 Calidad del tratamiento de datos

Está desarrollado en §4. En resumen:

- **Limpieza y preparación:** cinco trampas de formato de la entrega v2 corregidas al leer; 13 clones colapsados; fechas en dos formatos; universo simétrico por período de producción (§4.4, §4.6).
- **Faltantes y atípicos:** topes físicos y no cuantiles; ralentí separado de los viajes; faltantes estructurales tratados como información; el contador cortado, reconstruido desde el nivel del filtro (§4.5).
- **Consistencia y trazabilidad:** correcciones declaradas en YAML, holdouts congelados con huella, contrato por prefijo, chequeos automáticos, cada hallazgo con su comando (§4.9, Anexos A y B).

5.2.5 Claridad para mostrar los datos

- **En este informe,** una figura o tabla por idea y siempre con su fuente: la curva de detección contra falsas alarmas con el azar dibujado (figura 8), el perfil alineado al evento (figura 4), el riesgo por edad (figura 5), las celdas mercado × motor (figura 6), el puntaje hacia el evento (figura 9) y en qué se apoya el modelo (figura 10). Las figuras salen de un script con su YAML, desde las salidas de las corridas.
- **La demo de producto** (§5.2.1): calendario de la flota con una columna por semana, la ficha del auto con su puntaje contra el umbral y los hábitos contra la flota sana, y la perilla. Se revisó visualmente en notebook y en celular.
- **Dashboards internos:** el del análisis exploratorio (`scripts/dashboard_eda.py`, solo desarrollo) y el de explicabilidad y costos del finalista de la entrega 1 (`scripts/dashboard_k2/`).
- **La regla de presentación:** toda detección al lado del azar; “31 %” solo no dice nada, “31 % donde el azar da 7 %” sí.

5.2.6 Explicabilidad del modelo

Explicamos en dos niveles, y no los mezclamos.

1 · En qué se apoya la GRU (para Ford y el equipo técnico). Permutamos cada señal (sus 20 tramos juntos) o cada estática en la validación de cada fold y medimos cuánto cae la detección media al 5–20 % (figura 10).

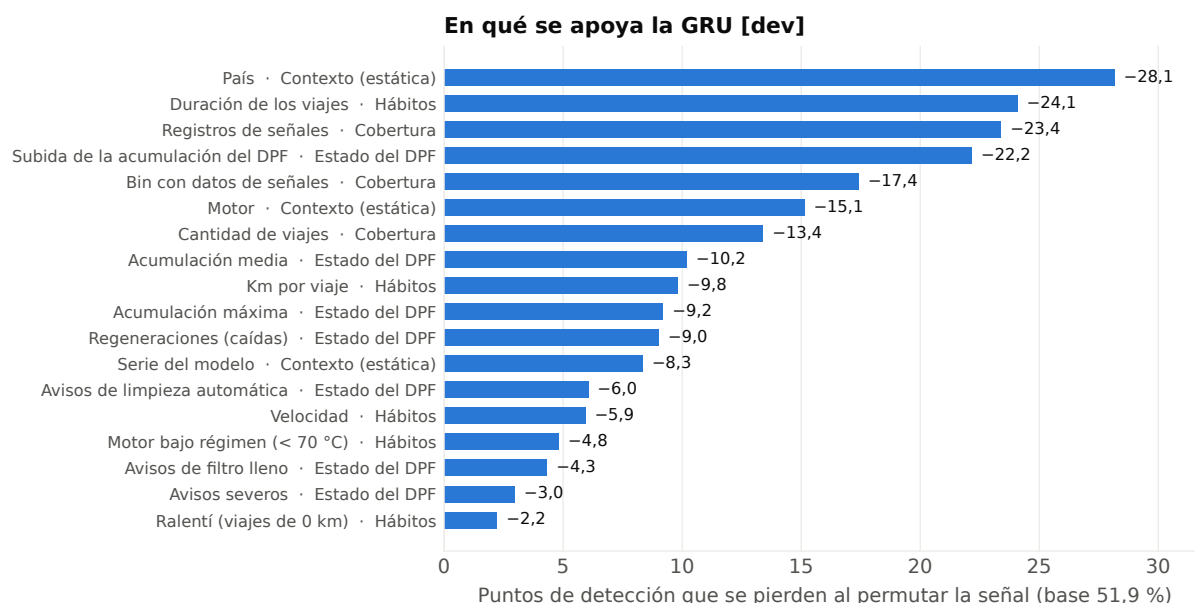


Figura 10: Caída de la detección media al 5–20 % al permutar cada señal en la validación (dev, semilla 42, repetición 0; detección base 51,9 %). Las caídas no se suman por familia: permutar una señal sola no es esconder la familia.

Fuente: `experiments/explain-gru-final/units.csv`; `configs/explain_gru_final.yaml` (`scripts/explain_perm_seq.py`).

- **Hábitos:** la duración de los viajes es el que más pesa (–24,1 puntos), por delante de km por viaje (–9,8), velocidad (–5,9), motor bajo régimen (–4,8) y ralentí (–2,2).
- **Estado del filtro:** cuánto sube la acumulación (–22,2), su nivel medio (–10,2) y máximo (–9,2), y las regeneraciones (–9,0).
- **Contexto:** el país pesa más que cualquier señal (–28,1), y el motor –15,1. Es coherente con la celda como piso: el riesgo no es el mismo en todos los mercados.
- **Cobertura:** cuántos registros y viajes hay en cada tramo (–23,4 y –13,4). Que un tramo tenga datos o no también informa.

La lectura física es la del DPF: viajes cortos que no dejan terminar la regeneración y hollín que se acumula. No le dimos ninguna regla del filtro: esa relación la aprendió de los datos.

2 · Por qué este auto (para el conductor y el concesionario). La GRU no tiene atribución por auto, y 7 de sus 15 canales son el estado del propio filtro, que no es algo que el conductor pueda cambiar. Por eso el porqué que se comunica *no* es lo que usó el modelo, sino una comparación descriptiva:

- el promedio de los cortes que dispararon la alerta se compara con la mediana de los autos sanos de desarrollo del mismo mercado;
- un hábito “se aparta” si supera al 75 % de esos sanos hacia el lado riesgoso, y se nombran hasta tres;
- **solo se nombran hábitos accionables cuyo efecto esperado coincide con la física del DPF**, de una lista cerrada antes de mirar los datos. Síntomas (nivel del filtro, regeneraciones, avisos, aceite, consumo) y contexto (mercado, odómetro) nunca llegan al conductor: van al taller.

El criterio físico viene de la explicabilidad del finalista de la entrega 1 (SHAP sobre survival stacking, reconstruido bit a bit): 10 de los 15 hábitos accionables con hipótesis física coincidían con la física del DPF; cuatro iban al revés (por ejemplo, viajes más largos asociados a más riesgo) y quedaron fuera del texto al cliente (`f4-explicabilidad-k2.md`). Por eso el texto dice “comparado

con autos sanos de tu zona”, nunca “porque”: el verificador rechaza el lenguaje causal.

5.2.7 El pitch

El pitch cuenta la misma historia que este informe, en siete bloques y 29 minutos, y con los mismos números (tabla 15). El hilo es un usuario común, “nuestro conductor”: hace viajes cortos en la ciudad, su filtro se tapa sin que nadie lo note, y al final de la historia recibe el aviso a tiempo.

Tabla 15: La estructura del pitch.

Bloque	Pregunta que responde	Frase ancla
1 · El problema	¿Por qué le importa a Ford?	“El auto avisa cuando el filtro ya no da más. Y el cliente se entera solo.”
2 · Los datos	¿Qué hay en la telemetría y qué trampas tenía?	“La forma de manejar deja huella meses antes.”
3 · El modelo	¿Qué probamos, qué ganó y cuánto detecta?	“Casi 1 de cada 3, tres meses antes, con 5 % de falsas alarmas.”
4 · El producto	¿Qué pasa después de la alerta? (agentes y demo en vivo)	“El modelo decide. El agente habla como una persona. El código cuida lo que dice.”
5 · Escalabilidad	¿Cómo llega a un millón de autos?	“Un millón de autos, sin un sensor nuevo.”
6 · Costos	¿Qué pierde el cliente hoy y cuánto cuesta evitarlo?	“Cada aviso a tiempo es un cliente que confía más en Ford.”
7 · Cierre	¿Por qué es distinto y qué necesita de Ford?	“Del testigo en el tablero al aviso a tiempo.”

Fuente: docs/pitch/presentacion-contenido.md; deck en docs/pitch/deck/index.html.

Las reglas para citar números son las mismas en el deck y acá: toda detección al lado del azar; el resultado en test y en rango; [test] y [dev] marcados; la anticipación como margen y no como pronóstico; nada de accuracy; y los límites dichos antes de que los pregunten.

6 Información complementaria

6.1 Documentación disponible

Qué recibimos. Un .zip con el dataset de telemetría y el diccionario de variables, en dos entregas (15-09 y 26-09-2026).

Qué hicimos con ella. Contrastamos el diccionario contra los archivos columna por columna (Anexo E): lo que faltaba, lo que venía con otro nombre y lo que no decía (escalas, formatos, semántica de las fechas). Todo lo que encontramos quedó escrito en el repositorio:

- docs/memoria/: un archivo por hallazgo, con el número y el comando que lo reproduce;
- docs/memoria/decisiones.md: qué se decidió, cuándo y por qué, la más nueva arriba;
- configs/: todas las corridas, paneles, holdouts y figuras como YAML versionados;
- results/: las métricas y la configuración completa de cada corrida que valió la pena.

6.2 Referentes técnicos

El desafío nombra como referentes técnicos a dos mentores del área de Ford. La solución de referencia de la tutora del equipo (una CNN-LSTM sobre las señales más el país) fue el primer baseline secuencial del proyecto y una de las familias medidas en test.

6.3 Lo que solo Ford tiene: preguntas abiertas

El límite del proyecto hoy no es el modelo: es la información. Estas son las preguntas cuya respuesta cambia los números, ordenadas por impacto:

Tabla 16: Lo que le pedimos a Ford y qué destraba cada pedido.

Pedido	Qué destraba
La tasa real de falla del DPF por país × motor en la flota	Separa la física de cómo se armaron las listas: es la duda más grande del proyecto. Si coincide con la de la muestra, motor y país entran sin reserva; si no, se reponderan las celdas
Qué eran los 278 fallados con fecha por defecto que salieron entre entregas, y con qué criterio se armó la lista de fallados	Explica por qué la tasa cambia tanto de un país a otro y por qué ningún fallado se produjo después de julio de 2025
Qué registra la fecha del evento de la entrega v2	Hoy la corremos 21 días a ojo, porque cae ~2 semanas después del taller
Más fallas registradas (hoy son 177)	La semilla mueve más que los hiperparámetros: más eventos bajan la varianza y suben el techo de información
Dónde se usa el auto, no dónde se vendió	La altura y el clima van en la dirección de la física, pero con la ciudad de venta no se separan del cero
Los costos reales (reparación, inspección, retención)	Convierten la perilla en una decisión con números de Ford
Un piloto en sombra de 3 meses en un país	La prevalencia real, las falsas alarmas reales y la detección realizada, sobre la flota completa

Fuente: f9-entrega-v2.md (“Lo que queda para preguntarle a Ford”); docs/pitch/presentacion-contenido.md bloque 7.

6.4 Trabajo futuro

- Reentrenar con los 557 autos y escribir el empaquetado del modelo (`export_model.py`).
- Validar en datos nuevos: una extracción posterior al 14-09-2026 de los mismos autos permitiría medir una mejora sin volver a usar el test.
- Medir en el piloto lo que estos datos no permiten: si los avisos cambian hábitos, cuántas fallas se evitan y cuánto consumo se ahorra.
- Explorar un punto de operación al 20 %: allí un ensamble heterogéneo (GRU + CNN-LSTM) sumó +6 puntos en desarrollo, sin llegar a la evidencia que pedía el preregistro.
- Extender el mismo circuito a otras fallas del postratamiento: el costo marginal de una segunda falla cubierta es bajo.

6.5 Conclusiones

1. **La telemetría que Ford ya recibe anticipa la falla del DPF:** comparando contra sanos del mismo mercado, el ralenti y el motor frío crecen hacia el evento, y se ven desde los primeros meses de uso.
2. **Leerla bien fue la mitad del trabajo:** cohortes muestreadas en períodos distintos, una fecha posterior al taller, un contador que se apaga y cinco trampas de formato hacían que cualquier modelo pareciera mejor de lo que es.
3. **La métrica importa tanto como el modelo:** el PR-AUC por fila se alcanzaba sin anticipar nada. La detección por auto, contra el azar y contra la regla “país × motor”, es la que mide lo que Ford necesita.

4. **La GRU finalista avisa a ~30 % de las fallas con 5 % de falsas alarmas**, y a ~50–60 % con 20 %, con unos tres meses de margen, en autos que nunca vio [\[test\]](#). Le duplica la detección a los modelos tabulares y de supervivencia, sin ganarle con evidencia a la celda.
5. **El producto convierte la alerta en una acción confiable**: un consejo en lenguaje humano, redactado por IA y verificado por código, con la perilla de falsas alarmas en manos de Ford.
6. **Lo que sigue lo destraba Ford**: la prevalencia real por país × motor, más eventos y un piloto en sombra.

Del testigo en el tablero al aviso a tiempo. Tu Ford te avisa antes.

A Reproducibilidad: comandos y configuraciones

Todo corre desde la raíz del repositorio con el entorno de `requirements.txt` (más `requirements-dl.txt` para las redes). Los datos no se versionan: la entrega v2 va en `data/v2/raw/` y se apunta con `FORD_DATA_DIR`. Sin red, `WANDB_MODE=disabled`. Cada número de este informe sale de alguna de estas líneas. La GRU no es reproducible bit a bit entre plataformas ni cantidades de hilos: una reproducción da números cercanos, no idénticos.

Datos, universo y panel

```
export FORD_DATA_DIR=$PWD/data/v2 WANDB_MODE=disabled OMP_NUM_THREADS=1
# entrega 1 vs v2
python scripts/compare_deliveries.py --config configs/data/compare_deliveries.yaml
# universo + holdout
python scripts/make_test_split.py --config configs/data/test_split.yaml --force
# cache del EDA (solo dev)
python scripts/build_eda_cache.py --config configs/data/eda_cache_v2.yaml
# EDA dentro del mercado
python scripts/eda_v2.py --config configs/eda_v2.yaml
# panel agregado (53 features)
python scripts/build_dataset.py --config configs/data/panel_v2.yaml
# + motor y serie
python scripts/build_dataset.py --config configs/data/panel_v2_estaticas.yaml
# folds 5 x 3, semilla 42
python scripts/make_splits.py --config configs/data/splits_panel_v2_r3.yaml
# secuencia 20 x 15
python scripts/build_seq_panel.py --config configs/data/panel_seq_trips_v2_estaticas.yaml
# chequeos del harness
python scripts/check_setup.py
```

El finalista en desarrollo

```
# semilla 42 (y _s1_r3, _s2_r3)
python scripts/train.py --config configs/exp_v2all_gru_trips_estaticas_r3.yaml
# ensamble por rango
python scripts/ensemble_rank.py --config configs/exp_v2all_seeds3_gru_trips_estaticas.yaml
# (a0), (a'), (b)
python scripts/audit_model.py --config configs/exp_v2all_gru_trips_estaticas_r3.yaml
# los 42 modelos, misma cuenta
python scripts/report_v2_models.py --config configs/report_v2_models.yaml
# anticipación en dev
python scripts/report_v2_leads.py --config configs/report_v2_leads.yaml
# sweep bayesiano (F10)
python scripts/sweep_gru.py --config configs/sweep_gru.yaml
# en qué se apoya
python scripts/explain_perm_seq.py --config configs/explain_gru_final.yaml
```

La medición en test

```
# el tiro preregistrado de F11
python scripts/eval_test.py --config configs/eval_test_f11.yaml --fit <yaml>
python scripts/eval_test.py --config configs/eval_test_f11.yaml --report
python scripts/eval_test.py --config configs/eval_test_f10_s42.yaml --fit
    ↪ configs/exp_v2all_gru_trips_estaticas_r3.yaml
# la GRU 42/1/2
python scripts/eval_test.py --config configs/eval_test_f10_s42.yaml --report
# todos los modelos
python scripts/eval_test.py --config configs/eval_test_modelos_v2.yaml --report
```

Este informe

```
# figuras y tablas generadas
python scripts/make_report_figures.py --config configs/report_figures.yaml
# PDF
cd docs/informe && tectonic informe.tex
```

Tabla 17: Las configuraciones que definen el finalista.

Archivo	Qué fija
configs/data/raw_sources.yaml	Esquema de los crudos y correcciones de la entrega v2
configs/data/vehicle_dedupe.yaml	Los 13 pares de clones
configs/data/test_split.yaml	Universo (ventana de producción) y sorteo del holdout
configs/data/panel_v2.yaml, panel_v2_estaticas.yaml	W, G, H, Δ , referencia – 21 d, emparejado de sanos
configs/data/features_v1.yaml	Las 53 features, umbrales y topes físicos
configs/data/panel_seq_trips_v2_estaticas.yaml	Los 15 canales y los 20 tramos de la secuencia
configs/exp_v2all_gru_trips_estaticas{,_s1,_s2}_r3.yaml	Arquitectura e hiperparámetros de la GRU, una por semilla
configs/exp_v2all_seeds3_gru_trips_estaticas.yaml	El ensamble por rango
configs/eval_test_f10_s42.yaml, eval_test_modelos_v2.yaml	La medición en test
configs/explain_gru_final.yaml	La explicabilidad por permutación
configs/report_figures.yaml	Las figuras y tablas de este informe

B Auditorías de leakage

Las tres defensas de construcción. (1) El gap de blanking de 500 km sobre la referencia corrida 21 días; (2) el split agrupado por vehículo, con los clones colapsados antes; (3) features solo hacia atrás y preprocesamiento ajustado por fold. Además, el corte en el fin de extracción de los sanos impide que la sola existencia de datos posteriores delate a un fallado.

Las auditorías obligatorias (`scripts/audit_model.py`), que toda corrida candidata tiene que pasar:

- **(a0) nulo global:** se permutan las features entre todas las filas y se reentrena. El PR-AUC tiene que caer a la tasa base; si no cae, hay leakage.
- **(a') aporte del cuándo:** se colapsa el puntaje de cada auto a su promedio, sin reentrenar. Lo que cae es lo que el modelo sabía del *cuándo*.

- **(b) atajo de calendario:** se le dan al modelo las columnas `aux_` de calendario (fechas de producción y venta, temperatura ambiente). Si el ROC sube más de 0,02, el modelo tiene un atajo disponible.
- **(a) permutación dentro del vehículo:** es *informativa* y *no es un nulo*. Conserva qué autos fallan, así que sube en vez de bajar; no aprueba nada.

Tabla 18: Auditorías de los modelos de la entrega v2 [dev] (semilla 42 de cada secuencial). Tasa base de la (a0): 0,058.

Modelo	(a0) PR-AUC	(a')	(b) calendario	Veredicto
GRU + viajes + estática (finalista)	0,061	+0,001	+0,004	aprueba
GRU + viajes	0,061	+0,003	+0,025: marca	aprueba
CNN-LSTM + viajes + estática	0,057	+0,009	+0,018	aprueba
CNN-LSTM + viajes	0,060	+0,026	+0,001	aprueba
CNN-LSTM de la tutora (señales + país)	0,056	+0,026	+0,028: marca	aprueba
Survival stacking + motor y serie	0,056	+0,025	+0,009	aprueba

Fuente: docs/memoria/f9-remedicion-completa-v2.md (auditorías). Survival stacking sin estática: (a') +0,016 ± 0,004 (f9-remedicion-v2.md).

Lectura. Ningún modelo filtra: con las features permutadas, todos caen a la tasa base. La (a') es positiva en todos los modelos de la v2 pero chica: lo que separa es sobre todo *qué auto*. El finalista no marca el atajo de calendario; la misma GRU sin estática sí lo marcaba (+0,025), igual que la CNN-LSTM de la tutora con solo señales.

La auditoría que llevó a cambiar la métrica. Con la primera entrega, el nulo global caía exacto a la tasa base en tres modelos (pipeline limpio), pero el nulo dentro del vehículo quedaba *por encima* de su propio modelo (0,1928 contra 0,1653 del control): permutar dentro del auto conserva qué autos fallan y le saca el ruido de qué ventana le tocó. Esa observación, junto con el techo de cohorte (Anexo F), es la razón por la que el finalista se elige por detección por auto, (a') y estabilidad, y no por PR-AUC por fila (decisiones.md, cierre de F3).

Una trampa de exposición que también auditamos. Con la primera entrega, los fallados conservaban 18,25 cortes por bolsa y los sanos 9,00: el tamaño de la bolsa solo, como puntaje, daba lift 1,79x. Desde entonces toda detección se compara contra un nulo que conserva el largo del historial (scripts/audit_mil_bagsize.py, scripts/audit_detection_null.py).

C Todos los modelos en test

Cada modelo se entrenó con todo el desarrollo (semilla 42, salvo los x3) y se midió con la misma cuenta: umbral exacto por presupuesto de falsas alarmas, alerta de dos cortes seguidos, 103 autos (32 fallados). Se midieron después del tiro preregistrado: describen, no eligen.

Tabla 19: Detección (%) en test por presupuesto de falsas alarmas [test]. "Prim." es la media al 5–20 %.

Modelo	2 %	5 %	10 %	15 %	20 %	25 %	30 %	Prim.	AUC auto	AUC celda	dev (5 · 10 · 20 %)
GRU x3 (finalista)	15,6	31,2	46,9	62,5	62,5	65,6	68,8	50,8	0,794	0,675	27 · 42 · 61
CNN-LSTM x3	3,1	15,6	56,2	56,2	68,8	68,8	68,8	49,2	0,790	0,638	22 · 41 · 62
Survival stacking	3,1	18,8	21,9	21,9	43,8	53,1	53,1	26,6	0,710	0,515	14 · 24 · 42
LightGBM (53 features)	0,0	15,6	25,0	28,1	34,4	53,1	56,2	25,8	0,725	0,619	18 · 28 · 42
Logística L2 (53 features)	0,0	6,2	12,5	31,2	34,4	46,9	53,1	21,1	0,679	0,556	15 · 23 · 37
SS + efecto aleatorio (GPBoost)	3,1	9,4	9,4	9,4	12,5	12,5	12,5	10,2	0,363	0,377	6 · 9 · 17
Celda mercado x motor (sin modelo)	0,0	0,0	21,9	21,9	50,0	50,0	50,0	23,4	—	—	18 · 33 · 62
Azar, media / p95 (nulo de la GRU)	3,4 / 9,4	7,0 / 15,6	13,7 / 25,0	18,6 / 31,2	24,4 / 40,6	28,6 / 43,8	33,9 / 50,0	—	—	—	—

Fuente: Detección y azar: experiments/test-modelos-v2/curve.csv (tabla generada); AUC y dev: docs/memoria/f11-test-resultado.md §3.

Comparaciones pareadas (bootstrap por vehículo, 2.000 réplicas, diferencia en puntos de primario):

- GRU – celda mercado × motor: +27,3, IC95 [−0,8; 46,1], $p(\text{dif} \leq 0) = 0,028$.
- Survival stacking – GRU: −24,2, IC95 [−43,0; −0,8] (no preregistrada).
- GRU 42/1/2 – GRU 101–103 (la misma configuración con otras semillas): +6,2, IC95 [−7,8; +18,0].
- GRU con etiqueta suave – GRU con etiqueta dura (el tiro preregistrado de F11, semillas 101–103 las dos): −13,3, IC95 [−25,0; +3,9].

Tabla 20: El tiro preregistrado de F11 [\[test\]](#): detección (%).

	2 %	5 %	10 %	15 %	20 %	25 %	30 %	Primario
GRU con etiqueta suave (candidato)	15,6	18,8	21,9	34,4	50,0	56,2	68,8	31,2
GRU con etiqueta dura (referencia, 101–103)	15,6	28,1	50,0	50,0	50,0	62,5	68,8	44,5
Celda mercado × motor	0,0	0,0	21,9	21,9	50,0	50,0	50,0	23,4

Fuente: docs/memoria/f11-test-resultado.md §1; experiments/test-f11/.

Notas. La CNN-LSTM es inestable a presupuestos bajos (15,6 % al 5 %, 56,2 % al 10 %). GPBoost invierte el orden: un auto nuevo entra sin su efecto aleatorio, y su AUC por auto ya estaba invertido en desarrollo (0,34). La celda detecta 0 % por debajo del 10 % porque alerta celdas enteras.

D Diccionario de features

Las 53 features agregadas (modelos tabulares y de supervivencia)

Cada feature se calcula sobre la ventana (corte – 1.000 km, corte] del odómetro, solo con viajes y señales anteriores al corte. Las `aux_` quedan en el panel para auditar y no entran a ningún modelo. Agregadores: mean/median/p25/max sobre los viajes de la ventana; `per_1000km`, conteo por 1.000 km recorridos; `per_day`, por día calendario; `half_*`, diferencia entre la segunda y la primera mitad de la ventana (tendencia); `gap_*`, distancia entre eventos; `km_since_last`, km desde el último evento; `any`, si ocurrió alguna vez.

Feature	Fuente	Columna derivada	Agregador
A · Térmica y trayectos cortos			
<code>feat_idle_frac</code>	<code>trips</code>	<code>idle</code>	mean
<code>feat_idle_per_1000km</code>	<code>trips</code>	<code>idle</code>	<code>per_1000km</code>
<code>feat_idle_frac_trend</code>	<code>trips</code>	<code>idle</code>	<code>half_mean_delta</code>
<code>feat_short_trip_frac_5km</code>	<code>trips</code>	<code>short_trip_moving</code>	mean
<code>feat_trips_below_regime_temp_frac</code>	<code>trips</code>	<code>below_regime</code>	mean
<code>feat_below_regime_moving_frac</code>	<code>trips</code>	<code>below_regime_moving</code>	mean
<code>feat_below_regime_frac_trend</code>	<code>trips</code>	<code>below_regime</code>	<code>half_mean_delta</code>
<code>feat_engine_temp_avg_median</code>	<code>trips</code>	<code>EngineTemperatureAvg_moving</code>	median
<code>feat_engine_temp_max_median</code>	<code>trips</code>	<code>EngineTemperatureMax_moving</code>	median
<code>feat_engine_temp_amplitude_mean</code>	<code>trips</code>	<code>engine_temp_amplitude_moving</code>	mean
<code>feat_coolant_temp_end_mean</code>	<code>trips</code>	<code>CoolantTemperatureEnd_moving</code>	mean
<code>feat_coolant_temp_start_median</code>	<code>trips</code>	<code>CoolantTemperatureStart</code>	median
<code>feat_cold_start_frac</code>	<code>trips</code>	<code>cold_start</code>	mean
<code>feat_chained_trip_frac</code>	<code>trips</code>	<code>chained</code>	mean
<code>feat_trip_distance_median_km</code>	<code>trips</code>	<code>trip_km_moving</code>	median
<code>feat_trip_distance_p25_km</code>	<code>trips</code>	<code>trip_km_moving</code>	p25
<code>feat_trip_duration_median_min</code>	<code>trips</code>	<code>trip_duration_min_moving</code>	median
B · Regeneración (DPF)			
<code>feat_regenerations_per_1000km</code>	<code>trips</code>	<code>regen_drop</code>	<code>per_1000km</code>

Feature	Fuente	Columna derivada	Agregador
feat_regenerations_trend_per_1000km	trips	regen_drop	half_count_delta_per_1000km
feat_distance_between_regen_mean_km	trips	regen_drop	gap_mean_km
feat_distance_between_regen_trend	trips	regen_drop	gap_trend_km
feat_km_since_last_regen	trips	regen_drop	km_since_last
feat_regen_residual_mean	trips	regen_residual	mean
feat_regen_start_level_mean	trips	regen_start_level	mean
feat_dpf_end_mean	trips	AirRegenerationEnd	mean
feat_dpf_end_max	trips	AirRegenerationEnd	max
feat_dpf_end_slope_per_1000km	trips	AirRegenerationEnd	half_slope_per_1000km
feat_dpf_positive_delta_frac	trips	dpf_positive_delta_moving	mean
feat_dpf_saturated_frac	trips	dpf_saturated_end	mean
feat_filter_abnormal_end_frac	trips	filter_abnormal_end	mean
feat_filter_cleaning_auto_end_frac	trips	filter_cleaning_auto_end	mean
feat_manual_regen_ever	trips	filter_manual	any
C · Uso			
feat_speed_kmh_mean	trips	speed_kmh_moving	mean
feat_trips_below_30kmh_frac	trips	urban_moving	mean
feat_moving_trips_per_1000km	trips	moving	per_1000km
feat_km_per_day	trips	trip_km	per_day
feat_trips_per_day	trips	moving	per_day
feat_hours_between_trips_median	trips	soak_min	median
aux_air_temp_avg	trips	AirTemperatureAvg	mean
aux_air_temp_min	trips	AirTemperatureMin	mean
D · Severidad (mensajes, aceite, consumo)			
feat_msg_full_per_1000km	signals	msg_full	per_1000km
feat_msg_overloaded_per_1000km	signals	msg_overloaded	per_1000km
feat_msg_overloaded_ever	signals	msg_overloaded	any
feat_msg_over_limit_per_1000km	signals	msg_over_limit	per_1000km
feat_msg_over_limit_ever	signals	msg_over_limit	any
feat_msg_at_limit_ever	signals	msg_at_limit	any
feat_msg_cleaning_auto_per_1000km	signals	msg_cleaning_auto	per_1000km
feat_msg_cleaning_auto_trend_per_1000km	signals	msg_cleaning_auto	half_count_delta_per_1000km
feat_msg_abnormal_frac	signals	msg_abnormal	mean
feat_msgs_per_1000km	signals	msg_any	per_1000km
feat_oil_life_mean	trips	oil_life	mean
feat_oil_life_slope_per_1000km	trips	oil_life	half_slope_per_1000km
feat_fuel_pct_per_100km_median	trips	fuel_pct_per_100km	median
aux_regen_marker_per_1000km	signals	regen_marker	per_1000km
Control de ventana (los emite windows.py)			
feat_n_trips_window	trips	*	count
feat_window_km_covered	trips	OdometerTripEnd	rango

Fuente: configs/data/features_v1.yaml (tabla generada); derivadas en src/features/trips.py y src/features/signals.py.

Los 15 canales de la secuencia (la GRU)

La ventana de 1.000 km se divide en 20 tramos de 50 km; un viaje cae en el tramo de su odómetro de fin. “zero” rellena con cero un tramo sin datos (los conteos); “carry” arrastra el último valor dentro de la misma ventana, siempre anterior al corte (un tramo sin viajes no es un tramo con ralentí cero).

Canal	Fuente	Columna	Agregado por tramo	Relleno
n_records	signals	*	count	zero · log1p
observed	signals	*	any	zero
acum_mean	signals	Acumulation	mean	carry
acum_max	signals	Acumulation	max	carry
acum_rise	signals	acum_rise	sum	zero
regen_drops	signals	acum_regen_drop	sum	zero
msg_full_frac	signals	msg_full	mean	zero
msg_severe_frac	signals	msg_severe	mean	zero
msg_cleaning_auto_frac	signals	msg_cleaning_auto	mean	zero
n_trips	trips	*	count	zero · log1p
idle_frac	trips	idle	mean	carry
below_regime_frac	trips	below_regime_moving	mean	carry
speed_kmh	trips	speed_kmh_moving	mean	carry
trip_km	trips	trip_km_moving	mean	carry
trip_duration_min	trips	trip_duration_min_moving	mean	carry

Fuente: configs/data/panel_seq_trips_v2_estaticas.yaml (tabla generada). Más país, motor y serie en one-hot como static_.

E Las tablas de Ford: anexo contra archivos

Tabla 22: *Trip Summary*: las 40 columnas del anexo 7.1 y lo que llegó.

Columna del anexo	Llegó	Uso
Vin	como VehicleCode	Identificador anonimizado
TripDatetimeStart/End	sí	Duración, reposo entre viajes, emparejado por mes (dos formatos mezclados)
OdometerTripStart/End	sí	Eje de km, km por viaje, ventanas
KilometerPerHour	sí	Nulo en los viajes de 0 km; se recalcula como km/duración
FuelLvlStartPc/EndPc	sí	Consumo como índice relativo (supera 100)
FuelLvlAutonomyStart	no	—
EngineOilLifePCStart/End	sí	Pendiente del nivel en la ventana (no cambia dentro del viaje)
EngineTemperatureMin/Max/Avg	sí	Bajo régimen (máx. < 70 °C), temperaturas
CoolantTemperatureStart/End	sí	Arranque en frío, refrigerante al terminar
DieselParticulateFilterStart/End	no, por ese nombre	Llega como AirRegenerationStart/End (= Acumulation, 99,8 %)
VehicleGPSLat/LongDataStart/End (4)	no	Sin GPS no hay ubicación ni altura por viaje
VehicleElevationRangeStart/End	no	Proxy probado: altura de la ciudad de venta
TirePressure{LF, RF, LR, RR}{Start, End} (8)	no	—
AirTemperatureStart/End/Min/Max/Avg	sí	Solo auditoría: deriva con el mes y el lugar
ManualRegenerationSootStart/End	no	Los mensajes de limpieza manual marcan el evento (rarísimos)
TripNumber	sí	—
No está en el anexo: AirFilterStart/End	sí	Nivel categórico del filtro (= Message, 99,97 %)

Fuente: docs/memoria/f2-diccionario-trips-incompleto.md, f2-columnas-airfilter-airregeneration.md; configs/data/raw_sources.yaml.

Tabla 23: *DynamicInformation y Vehicle General Information.*

Columna	Qué encontramos y cómo se usa
VehicleCode	Clave de las seis tablas; 13 vehículos con dos códigos
EventTimestamp	Llega como eventTimestamp, dos formatos mezclados
OdometerValue	Nulos concentrados en pocos autos; la ventana se ubica por el odómetro de los viajes
Acumulation	Nivel del filtro, escala 0–100 en pasos de 5; la misma variable que AirRegeneration
Message	9 niveles del filtro (“Normal Operation”, “Full”, “Overloaded”, “Over Limit”, limpiezas...); tasas por 1.000 km como severidad
Regenerations	El texto “Regeneration” marcando una fila; se corta el 25-05-2026: solo auditoría
DistanceBetweenRegenerations	Distancia al marcador anterior; hereda el corte: solo auditoría
VehCode	Como VehicleCode; en v2 los fallados traen una fila por evento
IdentificationDate	Días desde producción del evento; nula en todos los sanos: es la etiqueta. En los fallados de v2 llega renombrada
daysUntilSale	Solo auditoría (cohorte)
ProductionDay	En el eje del calendario con origen común (IQR 0 días); en los fallados de v2, +538 días; solo auditoría
Engine, ModelSeries	Candidatas en v2; se reporta siempre dentro de mercado × motor
SalesCountryCd	Llega como SalesCountry_cd; en v2 con código ISO
Nueva en v2: SalesCity	Altura de la ciudad de venta (probada, no entra)

Fuente: f1-datos-reales.md, f1-anclaje-temporal.md, f9-entrega-v2.md.

F Anexo técnico: el PR-AUC por fila y el techo de cohorte

Lo dejamos acá, y no en el cuerpo, porque en este panel no mide lo que Ford necesita: un PR-AUC por fila por debajo del techo de cohorte no demuestra anticipación.

Tabla 24: PR-AUC por fila en el dev v2 [dev], al lado del techo de cohorte.

	PR-AUC (lift)	ROC por fila	(a')
Tasa base	0,057 (0,97×)	0,489	−0,002
LightGBM (control)	0,112 (1,92×)	0,690	−0,019
Survival stacking	0,125 (2,14×)	0,709	+0,016
GRU + viajes + estática ×3	0,160	—	+0,006
Techo de cohorte (“¿este auto falla?”, sin nada del cuándo)	0,177 (3,03×)	—	—

Fuente: docs/memoria/f9-remedicion-v2.md; GRU: f9-remedicion-completa-v2.md (la (a') del ensamble es +0,006; la de la semilla 42 sola, +0,001).

Cómo se lee. Todas las filas positivas del panel están dentro de los cortes de autos que fallan. Puntuar cada fila con la cohorte del auto ya da 0,177; ningún modelo llega. Lo que un modelo agrega sobre ese techo se mide con (a') y con el PR-AUC restringido a los cortes de autos que fallan. Con la primera entrega el techo era 0,263 (2,10×), y la meta del plan inicial (lift de 1,6–2×) quedaba toda por debajo.